

République Tunisienne
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique

Université de Sfax

École nationale d'électronique
et des télécommunications de Sfax



Ingénieur en :
Génie Systèmes Électroniques
et Communication

Option :
Systèmes Embarqués

N° d'ordre : GEC-aaa-0-00-00

MEMOIRE

présenté à

**L'École Nationale d'Électronique et des
Télécommunications de Sfax**

en vue de l'obtention du

**Diplôme National d'Ingénieur en :
Génie Systèmes Électroniques et Communication**

**Option :
Systèmes Connectés**

par

Yessmine Ayédi

**Refonte du Back Office Frontend avec Développement d'un
Module de Recherche Intelligente**

Soutenu le 25/06/2025, devant la commission d'examen :

M. Gargouri Amir
Mme. Hentati Raida
Mme. Zghal Rim
M. Cheikhrouhou Amir
M. Drira Ahmed

Président
Rapporteur
Encadrante académique
Encadrant professionnel
Encadrant professionnel



DEDICACES



C'est grâce à dieu que ce travail a été réalisé. C'est grâce à Lui et à Sa volonté, ainsi qu'à la patience qu'Il m'a accordée tout au long de cette période de travail.

Avec l'expression de ma reconnaissance la plus profonde, je dédie ce projet de fin d'études :

À tous ceux qui m'ont soutenu(e),

À mes chers parents, Nassima et Taoufik,

Pour leurs efforts qui ont contribué à ma réussite. Quels que soient les mots employés, je ne pourrai jamais exprimer suffisamment ma sincère gratitude.

À mon frère, Omar,

Pour son soutien, je te souhaite tout le succès et le bonheur dans ta vie.

À toute ma famille élargie,

Merci d'avoir toujours été là, de m'avoir encouragé et d'avoir enrichi ma vie par votre amour et vos conseils.

À mes meilleurs amis et amies,

Pour leur encouragement et les merveilleux moments que nous avons partagés ensemble, ainsi que pour les précieux souvenirs que je chéris.

À vous tous,

Je dédie ce travail.

Yessmine AYEDI



REMERCIEMENTS

C'est avec un immense plaisir que j'exprime mes sincères remerciements et ma profonde gratitude à toutes les personnes qui ont contribué à l'élaboration de ce projet de fin d'études.

Je tiens à remercier chaleureusement **Mme Rim Zghal** pour sa disponibilité, sa patience, la qualité de son encadrement ainsi que pour ses précieux et judicieux conseils tout au long de ce travail.

J'exprime également ma reconnaissance à **M. Amir Cheikhrouhou** et **M. Ahmed Drira**, mes encadrants industriels, pour l'honneur qu'ils m'ont fait en acceptant de superviser ce projet. Leur disponibilité, leurs orientations pertinentes et la confiance qu'ils m'ont accordée ont été déterminantes dans la réussite de ce travail.

Je remercie également les honorables membres du jury pour le temps qu'ils m'ont consacré, ainsi que pour leurs remarques, leurs discussions enrichissantes et l'évaluation de ce modeste travail.

Enfin, j'adresse mes chaleureux remerciements à toute l'équipe de la société Platana pour leur accueil et leur soutien. Mes respects vont également à l'ensemble des enseignants de l'ENET'Com, dont la formation m'a permis d'acquérir des compétences solides en ingénierie.



TABLE DES MATIÈRES

LISTE DES FIGURES	vii
LISTE DES TABLEAUX	viii
LISTE DES ABRÉVIATIONS	ix
INTRODUCTION GÉNÉRALE	x
1 ETUDE PREALABLE	2
1.1 Introduction	3
1.2 Présentation de l'organisme d'accueil	3
1.2.1 Présentation Générale	3
1.2.2 Produits de Platana	4
1.2.3 Références de Platana	5
1.3 Cadre général du projet	6
1.3.1 Problématique et motivation	6
1.3.2 Etude de l'existant	7
1.3.2.1 Présentation de l'existant	7
1.3.2.2 Limitations de la solution existante	8
1.3.3 Solution proposée	9
1.4 Méthodologie adoptée et pilotage du projet	11
1.4.1 Méthodologie adoptée : Scrum	11
1.4.2 Planification des sprints : Backlog produit	13
1.4.3 Diagramme de Gantt	14
1.5 Conclusion	15
2 Spécification des besoins et choix techniques	16
2.1 Introduction	17
2.2 Spécification des besoins	17
2.2.1 Identification des acteurs	17

2.2.2	Identification des besoins fonctionnels	18
2.2.3	Diagramme de cas d'utilisation global	18
2.2.4	Identification des besoins non fonctionnels	19
2.3	Besoins techniques	20
2.3.1	Environnement matériel	20
2.3.2	Environnement logiciel et technologies utilisées	21
2.3.3	Framework utilisées	22
2.3.4	Outils de gestion de projets et de communication	23
2.4	Approches de développement	24
2.4.1	Approches pour le développement de l'interface utilisateur	24
2.4.1.1	Étude de l'approche Material-UI	24
2.4.1.2	Approche basée sur Syncfusion	25
2.4.1.3	Synthèse comparative et approche retenue	25
2.4.2	Approche de recherche basée sur l'intelligence artificielle	26
2.4.2.1	Étude de l'approche basée sur les LLM	27
2.4.2.2	Étude de l'approche NER	28
2.4.2.3	Synthèse comparative et approche retenue	29
2.4.3	Internationalisation i18n	29
2.5	Conclusion	30
3	Mise en place des fondations fonctionnelles	31
3.1	Introduction	32
3.2	Sprint 1 : Mise en œuvre de l'authentification et affichage des données	32
3.2.1	Backlog du sprint 1	32
3.2.2	Diagramme de cas d'utilisation du sprint 1	34
3.2.3	Diagrammes de séquence du sprint 1	37
3.2.4	Diagramme de classes du sprint 1	39
3.2.5	Réalisation	39
3.3	Sprint 2 : Ajout de filtres de recherche et composants UI	41
3.3.1	Backlog du sprint 2	41
3.3.2	Diagramme de cas d'utilisation du sprint 2	41
3.3.3	Diagrammes de séquence du sprint 2	45
3.3.4	Diagramme de classes du sprint 2	48
3.3.5	Réalisation	48
3.4	Conclusion	51

4	Mise en œuvre de la recherche intelligente et personnalisation de l'application SAV	52
4.1	Introduction	53
4.2	Sprint 3 : Mise en place de l'intelligence artificielle pour la recherche des dossiers SAV	53
4.2.1	Backlog du sprint 3	53
4.2.2	Identification des plateformes d'intégration des modèles LLM	54
4.2.3	Présélection des modèles LLM pour la recherche intelligente	55
4.2.4	Configuration du prompt engineering	55
4.2.5	Tests réalisés et choix du modèle LLM	56
4.2.6	Paramètres ajustables du modèle retenu	57
4.2.7	Tests réalisés et choix de la plateforme d'intégration API	58
4.2.8	Résultats obtenus du modèle LLaMA 70B-8192	59
4.3	Sprint 4 : Intégration de la recherche intelligente et optimisation multilingue .	61
4.3.1	Backlog du sprint 4	61
4.3.2	Diagramme du cas d'utilisation du sprint 4	62
4.3.3	Diagrammes de séquence du sprint 4	65
4.3.4	Diagramme de classes du sprint 4	68
4.3.5	Réalisation	68
4.4	Conclusion	70
	CONCLUSION GÉNÉRALE	71
	WEBOGRAPHIE	72

LISTE DES FIGURES

1.1	Logo de la société « PLATANA »	3
1.2	Références de Platana	5
1.3	Interface de recherche des dossiers sav	8
1.4	Processus de gestion des recherches	10
1.5	Processus Scrum	11
1.6	Diagramme de Gantt de notre projet	15
2.1	Diagramme de cas d'utilisations général	19
2.2	Logo IntelliJ IDEA	21
2.3	Logo Postman	21
2.4	Logo TypeScript	22
2.5	Logo Keycloak	22
2.6	Logo Spring boot	23
2.7	Logo Angular	23
2.8	Logo Draw.io	23
2.9	Logo GitLab	24
2.10	Logo Material UI	25
2.11	Logo Syncfusion	25
2.12	Large Language Models	27
2.13	Extraction d'entités à partir d'un texte	28
3.1	Processus d'authentification et affichage des dossiers SAV	33
3.2	Diagramme du cas d'utilisation du sprint 1	34
3.3	Diagramme de séquence du cas d'utilisation « S'authentifier »	37
3.4	Diagramme de séquence du cas d'utilisation « Afficher la liste des dossiers SAV »	38
3.5	Diagramme de classes du sprint 1	39
3.6	Interface d'authentification avec keycloak	40
3.7	Interface de la liste des dossiers SAV affichés dans une grid syncfusion	40
3.8	Diagramme de cas d'utilisation du sprint 2	42

3.9	Diagramme de séquence du cas d'utilisation « Rechercher des dossiers SAV » .	46
3.10	Diagramme de séquence du cas d'utilisation « Naviguer dans les résultats» . . .	46
3.11	Diagramme de séquence du cas d'utilisation «Personnaliser l'interface»	47
3.12	Diagramme de séquence du cas d'utilisation «Accéder à la page de création des dossiers»	47
3.13	Diagramme de classes du sprint 2	48
3.14	Interface de recherche des dossiers	49
3.15	Interface de recherche intégrant une barre de saisie	49
3.16	Interface de recherche avec filtres avancés	50
3.17	Personnalisation de la grille	50
3.18	Interface de sélection des filtres additionnels via 'Plus'	51
4.1	Logo Groq	54
4.2	Logo OpenRouter	55
4.3	Réponse du modèle LLaMA3-70B-8192	56
4.4	Réponse du modèle DeepSeek R1 Distill Llama 70B	57
4.5	Logs de génération du modèle LLaMA3-70B-8192 via Groq	58
4.6	Logs de génération du modèle LLaMA3-70B-8192 via openRouter	59
4.7	Exemple de requête en français avec fautes d'orthographe saisie par un utilisateur	59
4.8	Réponse du modèle à une requête en français contenant des fautes d'orthographe	60
4.9	Requête formulée en espagnol par l'utilisateur	60
4.10	Réponse du modèle à une requête en espagnol	60
4.11	Requête demandant les dossiers avec une date de création spécifique	61
4.12	Réponse du modèle avec conversion précise de la date en timestamps	61
4.13	Diagramme de cas d'utilisation du sprint 4	62
4.14	Diagramme de séquence du cas d'utilisation «Recherche avec filtre IA»	66
4.15	Diagramme de séquence du cas d'utilisation «Combinaison de recherche intelligente et filtres classiques»	66
4.16	Diagramme de séquence du cas d'utilisation «Changer la langue de l'interface»	67
4.17	Diagramme de séquence du cas d'utilisation «Basculer entre les organisation» .	67
4.18	Diagramme de classes du sprint 4	68
4.19	Interface de combinaison de recherche intelligente et filtres classiques	69
4.20	Interface de sélection de la langue	69
4.21	Interface de sélection des organisations	70



LISTE DES TABLEAUX

1.1	Solutions proposées par Platana	4
1.2	Équipe Scrum	13
1.3	Backlog produit	13
2.1	Environnement matériel utilisé	20
2.2	Comparaison des fonctionnalités de Material-UI et Syncfusion	26
2.3	Comparaison entre LLM et NER	29
3.1	Backlog du sprint 1	32
3.2	Description textuelle du cas d'utilisation « S'authentifier »	34
3.3	Description textuelle du cas d'utilisation « Afficher la liste des dossiers SAV »	36
3.4	Backlog du sprint 2	41
3.5	Description textuelle du cas d'utilisation « Rechercher les dossiers SAV »	43
3.6	Description textuelle du cas d'utilisation « Naviguer dans les résultats »	44
3.7	Description textuelle du cas d'utilisation « Personnaliser l'interface »	44
3.8	Description textuelle du cas d'utilisation « Accéder à la page de création des dossiers »	45
4.1	Backlog du sprint 3	53
4.2	Backlog du sprint 4	62
4.3	Description textuelle du cas d'utilisation « Rechercher les dossiers avec IA »	63
4.4	Description textuelle du cas d'utilisation « Combiner la recherche intelligente avec des filtres avancés »	63
4.5	Description textuelle du cas d'utilisation « Changer la langue de l'interface »	64
4.6	Description textuelle du cas d'utilisation « Basculer entre les organisations »	65



LISTE DES ABRÉVIATIONS

API Application Programming Interface

CSS Cascading Style Sheets

HTTP Hypertext Transfer Protocol

IA Intelligence artificielle

IDE Integrated Development Environment

JWT JSON Web Tokens

LLM Large Language Model

LLaMA Large Language Model Meta AI

NER Named Entity Recognition

NLP Natural Language Processing

SAV Service Après Vente

Sass Syntactically Awesome Stylesheets

TS Type Script

UML Unified Modeling Language

UI User Interface



INTRODUCTION GÉNÉRALE

Aujourd'hui, les entreprises évoluent dans un environnement technologique en constante transformation. Les avancées en intelligence artificielle, la digitalisation des services après-vente (SAV) et les attentes croissantes des clients imposent une réinvention des outils traditionnels pour rester compétitives.

Dans ce contexte, la gestion des dossiers SAV devient un enjeu stratégique. Une prise en charge inefficace peut entraîner des retards, des erreurs et un fort taux d'insatisfaction. En procédant à une refonte des interfaces existantes et en intégrant des solutions d'intelligence artificielle (IA), les entreprises peuvent optimiser leurs processus internes, renforcer l'expérience utilisateur et accroître leur efficacité opérationnelle. C'est dans cette optique que s'inscrit ce projet de fin d'études, réalisé au sein de l'entreprise Platana, dans le cadre de l'obtention du diplôme national d'ingénieur.

L'objectif principal de ce travail est de concevoir une nouvelle interface dédiée à la gestion des dossiers SAV chez Platana, intégrant une fonctionnalité de recherche intelligente fondée sur l'IA. La solution repose sur une architecture technologique moderne s'appuyant sur Angular 18 et les composants Syncfusion, ainsi qu'une recherche sémantique exploitant un modèle de langage. Elle vise à offrir aux opérateurs SAV une interface intuitive, rapide et multilingue, facilitant la recherche et la gestion des dossiers, tout en garantissant une résolution efficace des demandes dans les délais impartis.

La refonte, associée à l'intelligence artificielle, apporte plusieurs bénéfices concrets. Elle améliore l'ergonomie globale, réduit les temps de recherche grâce à l'interprétation sémantique des requêtes, et s'adapte aux exigences d'une clientèle internationale. En adoptant cette solution

innovante, Platana renforce ses performances, recentre ses équipes sur des tâches à forte valeur ajoutée, et consolide sa position dans un secteur SAV omnicanal de plus en plus exigeant.

Le présent rapport est structuré en quatre chapitres. Le premier chapitre présente l'organisme d'accueil, Platana, et définit le cadre général du projet, en analysant les limites de l'interface existante et en proposant une démarche de refonte basée sur la méthodologie agile Scrum. Le deuxième chapitre décrit les besoins fonctionnels et non fonctionnels, précise les choix techniques adoptés et justifie les technologies retenues. Le troisième chapitre est consacré à la mise en œuvre des fondations fonctionnelles, telles que l'authentification via Keycloak ou le développement des composants d'interface, y compris la recherche avec filtres. Enfin, le quatrième chapitre porte sur l'intégration de la recherche intelligente, incluant l'utilisation de modèles LLM, le prompt engineering et la gestion multilingue avec i18n.

ETUDE PREALABLE

Sommaire

1.1	Introduction	3
1.2	Présentation de l'organisme d'accueil	3
1.2.1	Présentation Générale	3
1.2.2	Produits de Platana	4
1.2.3	Références de Platana	5
1.3	Cadre général du projet	6
1.3.1	Problématique et motivation	6
1.3.2	Etude de l'existant	7
1.3.3	Solution proposée	9
1.4	Méthodologie adoptée et pilotage du projet	11
1.4.1	Méthodologie adoptée : Scrum	11
1.4.2	Planification des sprints : Backlog produit	13
1.4.3	Diagramme de Gantt	14
1.5	Conclusion	15

1.1 Introduction

Dans ce chapitre intitulé Étude préalable, nous posons les bases du projet en explorant le contexte et les fondations nécessaires à sa compréhension. Tout d'abord, nous présentons l'organisme d'accueil, l'entreprise Platana, en détaillant ses activités, ses produits et ses références, afin de mieux comprendre son positionnement et ses enjeux. Ensuite, nous abordons le cadre général du projet, en précisant la problématique, les motivations et les limites de la solution actuelle utilisée pour la gestion des dossiers du Service Après-Vente (SAV). Nous proposons également une solution innovante visant à répondre à ces défis. Enfin, nous décrivons la méthodologie adoptée, basée sur l'approche agile Scrum, pour garantir une gestion efficace du projet.

1.2 Présentation de l'organisme d'accueil

Dans cette section, nous examinons de près l'entreprise d'accueil ainsi que les produits qu'elle propose, offrant ainsi une vue d'ensemble complète de ses activités et de son offre.

1.2.1 Présentation Générale

Notre projet de fin d'études s'est déroulé au sein de l'entreprise Platana [1], une société française de conseil et de services informatiques fondée le 15 septembre 2017. Implantée à Paris, Lille et Sfax, Platana est spécialisée dans le conseil en systèmes et logiciels informatiques. Elle propose une plateforme après-vente qui assure un accompagnement complet de ses clients tout au long du cycle de vie de leurs produits. Grâce à une équipe d'experts qualifiés, Platana prend en charge les demandes en omnicanal, du premier diagnostic jusqu'à la résolution, en optimisant l'expérience client et l'efficacité opérationnelle.



FIGURE 1.1 – Logo de la société « PLATANA »

Juin 2024 : Platana remporte le Trophée « LSA 100% Omnicanal », dans la catégorie « *Meilleur SAV* ».

L'entreprise permet aux retailers de transformer leur Service Après-Vente (SAV) en offrant :

- Une expérience client omnicanale et fluide
- Des collaborateurs efficaces grâce à des process digitalisés
- Un refinancement optimisé des garanties produit auprès des fabricants
- Une meilleure durabilité des produits

1.2.2 Produits de Platana

Platana propose plusieurs solutions logicielles permettant d'optimiser la gestion du Service Après-Vente (SAV), la réparation et le rappel produit.

TABLE 1.1 – Solutions proposées par Platana

Solution	Description
Rappel Produit	Permet de déployer et piloter des campagnes de rappel produit avec efficacité et rigueur. Cette solution assure la gestion, le suivi et le traitement de toutes les étapes d'un rappel produit en proposant des fonctionnalités adaptées aux exigences du marché : campagnes omnicanal (mail, téléphone, courrier, SMS, retail), autonomie dans le choix de la solution adaptée, dashboard et historisation.
Gestion de la relation client (CRM)	Offre une gestion optimisée et centralisée des interactions avec les clients afin d'améliorer leur satisfaction et leur fidélisation.
Réparation	Facilite la supervision des réparations en atelier ou à domicile, en optimisant la gestion des équipes internes et des partenaires

Logiciel de gestion des prestations	Solution permettant une gestion efficace des services et interventions, assurant une meilleure traçabilité et une amélioration des performances opérationnelles.
Service Après-Vente	Digitalise les procédures après-vente afin d’optimiser l’expérience des clients et des collaborateurs en magasin.
Logiciel de gestion de l’expérience client	Plateforme conçue pour analyser et améliorer en continu l’expérience client après-vente.

1.2.3 Références de Platana

La présentation des références de l’entreprise, fondée sur une sélection rigoureuse et prestigieuse de clients, témoigne de la reconnaissance et de la confiance accordées à Platana. La figure 1.2 illustre les références de Platana.



FIGURE 1.2 – Références de Platana

1.3 Cadre général du projet

Ce travail s'inscrit dans le cadre du stage de fin d'études, en vue de l'obtention du diplôme national d'ingénieur en électronique de communication. L'objectif principal du projet est de réaliser une refonte complète de l'interface existante dédiée à la gestion des dossiers du Service Après-Vente (SAV) chez Platana, tout en intégrant une fonctionnalité de recherche intelligente basée sur l'intelligence artificielle (IA). Ce module, conçu pour les clients de Platana, vise à doter ses magasins d'un outil moderne et performant, facilitant les interactions avec la clientèle et assurant un service après-vente complet et efficace. Le projet englobe des phases de recherche, de conception, et d'implémentation de fonctionnalités avancées, en s'appuyant sur des technologies de pointe telles que Angular 18 [2], Syncfusion [3], et le modèle LLaMA 70B-8192 [4] via l'API Groq [5], afin d'optimiser l'expérience utilisateur et de répondre aux exigences spécifiques du marché.

1.3.1 Problématique et motivation

Aujourd'hui, le service après-vente (SAV) joue un rôle stratégique dans la fidélisation et la satisfaction des clients. Il ne se limite plus à une simple extension du processus de vente, mais devient un pilier essentiel de l'expérience utilisateur globale. Dans un marché extrêmement concurrentiel, où les consommateurs sont de plus en plus informés et exigeants, la qualité du SAV influe directement sur la réputation et la pérennité des entreprises. Pour une société comme Platana, proposer un outil performant, moderne et intuitif n'est plus un avantage compétitif, mais une nécessité. Cet outil doit être capable de traiter efficacement les demandes, tout en garantissant une expérience utilisateur fluide et agréable. L'interface actuelle, construite sur Angular 10 [6] et utilisant Material-UI [7], montre aujourd'hui ses limites : des capacités de recherche trop classiques face à des volumes de données croissants, un manque de flexibilité dans la personnalisation des composants, et une ergonomie perfectible, avec un design peu intuitif. C'est dans ce contexte qu'une refonte complète de l'interface s'impose. Celle-ci vise à moderniser l'environnement technologique, à améliorer l'ergonomie globale, et à adopter

Syncfusion [3] à la place de Material-UI [7]. Ce choix permet de bénéficier de composants UI plus riches, interactifs et mieux adaptés à des cas d'usage professionnels complexes. Mais l'évolution majeure réside dans l'intégration d'une recherche intelligente basée sur l'intelligence artificielle, qui transforme en profondeur l'expérience utilisateur. Grâce à l'utilisation d'un grand modèle de langage (LLM)[8], cette IA est capable de comprendre l'intention derrière les requêtes utilisateur, de suggérer des filtres pertinents, et d'adapter dynamiquement les résultats affichés. Cette capacité sémantique représente une avancée majeure par rapport aux filtres manuels classiques. Elle réduit le temps de recherche, améliore la précision des résultats, et offre un dialogue plus naturel entre l'utilisateur et le système. En s'appuyant sur cette intelligence, Platana renforce non seulement l'efficacité de son SAV, mais affirme également sa capacité d'innovation, en anticipant les besoins de ses utilisateurs avec des technologies de pointe.

1.3.2 Etude de l'existant

Dans le cadre de ce projet, l'étude de l'existant a permis de mettre en lumière les limites de l'application actuelle, tant sur le plan technique que fonctionnel. Cette analyse constitue une base essentielle pour justifier la nécessité d'une refonte complète et guider les choix de conception de la nouvelle version.

1.3.2.1 Présentation de l'existant

L'application développée propose une interface intuitive et fonctionnelle, facilitant la consultation et la recherche des dossiers SAV via un tableau de données clair, structuré et ergonomique. Pour optimiser l'accès à l'information, deux modes de recherche complémentaires sont proposés :

- Recherche filtrée : cette fonctionnalité permet de trier les dossiers selon des critères prédéfinis tels que les dates, les statuts ou d'autres attributs spécifiques, assurant une navigation ciblée et efficace.
- Barre de recherche simple : dotée d'une interface conviviale, cette option autorise la recherche par mots-clés (ex. : numéro de dossier, nom ou prénom du créateur), permettant aux utilisateurs de retrouver rapidement les dossiers pertinents.

Sur le plan technologique, l'application repose sur le framework Angular 10 [6] et la bibliothèque Material UI [7], offrant une interface moderne, responsive et adaptée à une grande diversité d'interactions utilisateur. La figure 1.3 présente cette interface.

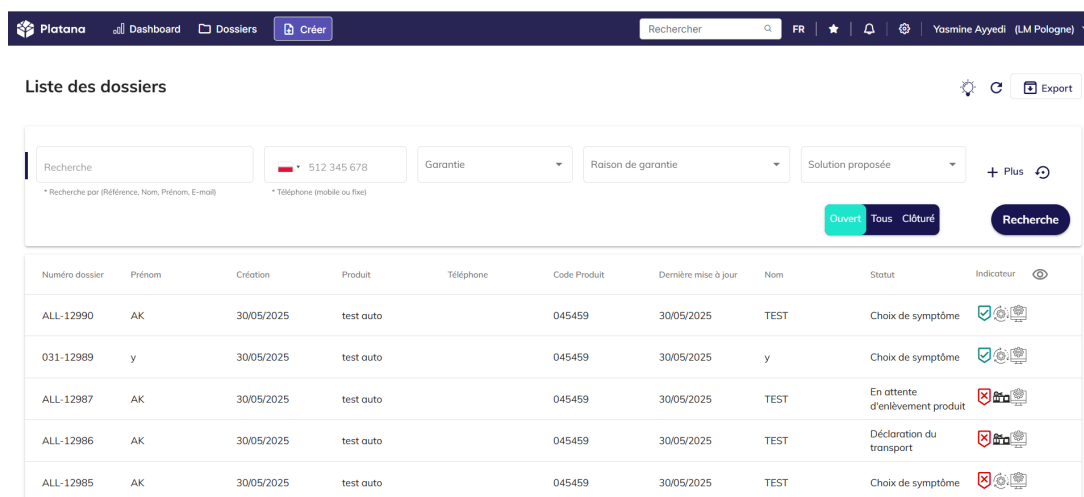


FIGURE 1.3 – Interface de recherche des dossiers sav

1.3.2.2 Limitations de la solution existante

L'étude de l'existant a révélé plusieurs limitations techniques et fonctionnelles, affectant la performance, l'ergonomie et la capacité de l'application à répondre aux besoins actuels des utilisateurs. Ces limitations sont les suivantes :

- Limitations technologiques : Angular 10 [6] est aujourd'hui obsolète, ne bénéficiant plus des mises à jour récentes en matière de sécurité, de performance et de compatibilité avec les bibliothèques modernes. L'interface repose sur plusieurs bibliothèques UI non unifiées, ce qui engendre des conflits de styles fréquents, un temps de développement allongé dû à la multiplicité des dépendances, et une cohérence visuelle insuffisante.
- Limitations de l'expérience utilisateur (UX/UI) : L'interface actuelle souffre de limites en ergonomie et intuitivité, notamment en raison de l'usage de Material-UI [7]. Bien que cette bibliothèque offre de nombreux composants prêts à l'emploi, son intégration génère plusieurs contraintes. Les composants de base (boutons, formulaires, grilles) manquent d'interactivité native, ce qui les rend peu adaptés aux scénarios métiers complexes. Leur

personnalisation requiert des ajustements manuels, allongeant les délais de développement et compliquant la maintenance. De plus, l'usage combiné de plusieurs bibliothèques UI provoque une fragmentation technique, engendrant conflits, surcharge stylistique et manque de cohérence visuelle, au détriment de l'expérience utilisateur

- Limites fonctionnelles de la recherche : Le système de recherche intégré à l'application présente des contraintes significatives, notamment l'absence d'un moteur de recherche intelligent. Les utilisateurs sont ainsi obligés d'entrer des mots-clés exacts pour obtenir des résultats pertinents, ce qui limite l'efficacité dans la gestion des dossiers. De plus, aucune tolérance aux erreurs n'est prise en compte : les fautes de frappe, les synonymes ou les formulations libres ne sont pas reconnues, rendant l'outil rigide face à des requêtes variées. La barre de recherche, dans sa forme actuelle, reste très basique et ne repose pas sur l'interprétation du sens ou de l'intention de l'utilisateur, ce qui limite sa capacité à répondre de manière adaptée aux besoins spécifiques des dossiers SAV

1.3.3 Solution proposée

Afin de dépasser les limites des systèmes actuels de gestion des dossiers SAV chez Platana, nous concevons et développons une solution digitale innovante, spécifiquement pensée pour répondre aux exigences fonctionnelles, ergonomiques et techniques de l'entreprise.

Donc la solution proposée s'articule autour de deux axes majeurs :

- La refonte complète de l'interface utilisateur existante.
- L'intégration d'un moteur de recherche intelligent multilingue, basé sur l'IA.

La refonte de l'interface a conduit à une modernisation radicale de la plateforme SAV, reposant sur les technologies web les plus avancées. Nous adoptons Angular 18 [2] pour une architecture robuste et intégrons les composants Syncfusion [3], offrant une interface moderne, fluide, réactive et interactive. Des filtres avancés, comme des menus déroulants dynamiques, des champs multisélection et une grille de résultats interactive, facilitent l'exploration et le traitement des dossiers. Cette solution garantit une expérience intuitive, efficace et conforme aux standards actuels du web, redéfinissant l'interaction avec les données SAV.

Parallèlement, nous intégrons un moteur de recherche intelligent, propulsé par le modèle LLM [8] de dernière génération LLaMA3-70B-8192 [4]. Renommé pour ses capacités en compréhension du langage naturel, ce modèle interprète avec précision les requêtes formulées librement, même imprécises ou multilingues. Il les traduit en filtres structurés qu'il injecte automatiquement dans la grille de résultats, assurant un affichage instantané, pertinent et contextualisé. Cette intelligence multilingue élimine les barrières linguistiques, rend la plateforme accessible à tous, y compris aux utilisateurs non techniques, et optimise la rapidité et la pertinence des réponses.

La force de cette solution réside dans son approche hybride, combinant une recherche classique via des filtres dynamiques pour un contrôle manuel précis, avec une recherche intelligente assistée par IA, capable de déduire les intentions sous-jacentes des requêtes naturelles. Cette complémentarité répond aux besoins opérationnels, s'adapte aux préférences des opérateurs et renforce la flexibilité du système. Grâce à cette intégration, la plateforme se distingue par une productivité accrue, une expérience fluide et personnalisée, une ergonomie optimisée, une navigation simplifiée et un traitement accéléré des requêtes, positionnant l'entreprise à la pointe de l'innovation dans le SAV omnicanal. Le processus de gestion des recherches est synthétisé dans la figure 1.4

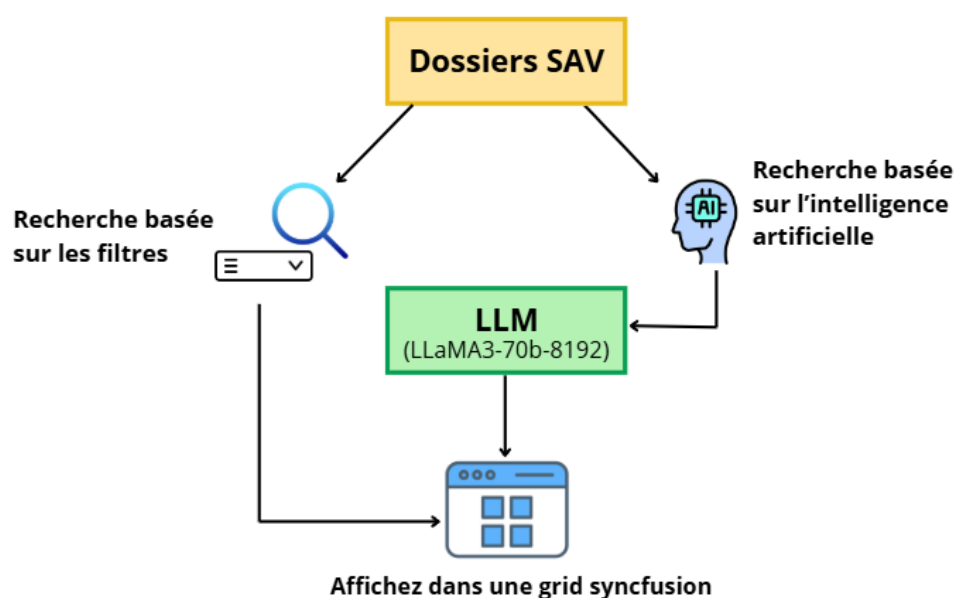


FIGURE 1.4 – Processus de gestion des recherches

1.4 Méthodologie adoptée et pilotage du projet

Dans cette partie nous présentons la méthodologie adoptée durant notre projet.

1.4.1 Méthodologie adoptée : Scrum

Les méthodologies et techniques de gestion de projet sont déterminantes pour la réalisation des objectifs, en assurant une structuration rigoureuse et cohérente du projet.

Parmi les différentes méthodes, Scrum [9] se démarque comme une approche agile largement adoptée dans le secteur du développement logiciel. Conçue pour les petites équipes auto-organisées et les projets modulaires, elle favorise une amélioration continue du produit grâce à des retours fréquents des utilisateurs finaux, tout en renforçant la productivité des équipes.

Au-delà de sa flexibilité face à l'évolution des besoins, Scrum permet également d'étendre l'agilité à l'échelle de l'organisation. Elle s'inscrit pleinement dans les principes fondamentaux du Manifeste Agile, en mettant l'accent sur la collaboration avec le client, l'adaptabilité, les interactions humaines, ainsi que la livraison régulière de solutions fonctionnelles.

La figure 1.5 ci-dessous présente les différents rôles au sein d'une équipe Scrum ainsi que les principaux événements qui rythment le déroulement du processus.

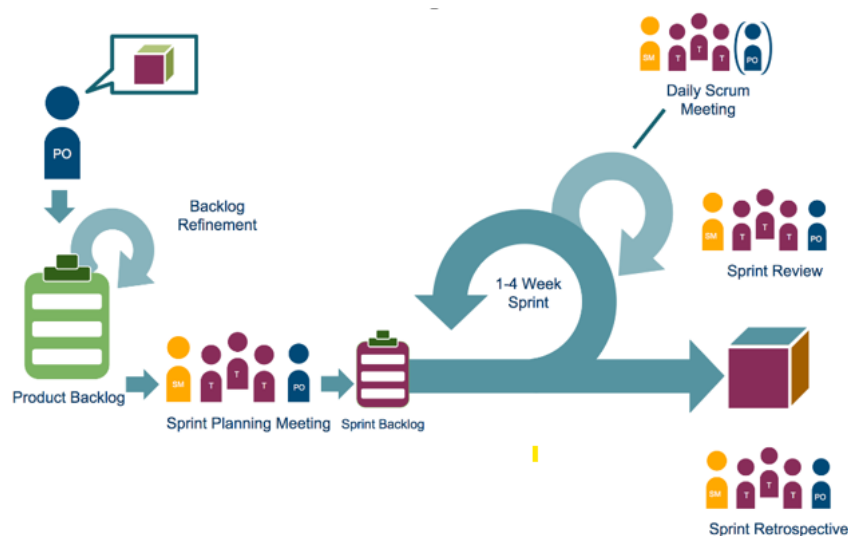


FIGURE 1.5 – Processus Scrum

L'adoption de la méthodologie Scrum pour le pilotage de notre projet répond à un objectif précis : mettre en place un cadre de travail performant reposant sur des itérations courtes, appelées "Sprints", facilitant ainsi la gestion de projets complexes.

Ce cadre s'appuie sur trois principes fondamentaux :

- Transparence : L'ensemble des membres de l'équipe a un accès constant aux informations liées au produit en développement, ce qui renforce la communication et la compréhension collective.
- Inspection : Des points de contrôle réguliers permettent d'évaluer l'avancement du projet et d'identifier d'éventuels ajustements à apporter, favorisant ainsi l'amélioration continue.
- Adaptation : À partir des observations issues des inspections, des actions correctives sont mises en œuvre pour ajuster le projet aux besoins évolutifs. L'agilité constitue ainsi le cœur de cette démarche.

Le choix de Scrum en tant que méthodologie de pilotage de notre projet repose sur l'objectif clair et précis d'établir un cadre de travail efficace en utilisant des itérations courtes appelées "Sprints", ce qui facilite la gestion de projets complexes.

La méthode agile Scrum repose sur trois rôles clés, complémentaires et indispensables au bon déroulement du projet. Il est crucial de bien comprendre les responsabilités associées à chacun :

- Le Scrum Master (SM) : véritable facilitateur, il adopte une posture de servant leader en soutenant l'équipe et en favorisant un environnement propice à la collaboration et à la création de valeur.
- Le Product Owner (PO) : il fait le lien entre les parties prenantes métier et l'équipe technique. Porteur de la vision produit, il est responsable de la gestion et de la priorisation du backlog.
- L'équipe de développement : elle est chargée de transformer les besoins exprimés en fonctionnalités concrètes et opérationnelles.

Notre équipe Scrum se compose de trois membres, dont les rôles sont présentés dans le tableau 1.2 ci-dessous .

TABLE 1.2 – Équipe Scrum

Rôle	Nom	Description
Stagiaire	Yessmine Ayedi	Développement de l'application
Encadrant (Scrum Master)	Amir cheikh Rouhou	Expert technique et fonctionnel, supervision du développement
Product Owner	Hela elloumi	Spécification des besoins, validation des incréments

1.4.2 Planification des sprints : Backlog produit

La méthode Scrum propose de découper le projet en une série de sprints. En tant que méthode agile, chaque sprint est limité dans le temps à un maximum de 30 jours.

Selon la méthodologie Scrum, au début de chaque sprint, une réunion de planification a lieu, au cours de laquelle les membres de l'équipe créent un backlog de sprint. Pour notre solution, nous avons décidé de réaliser 5 sprints, comme indiqué dans le tableau 1.3.

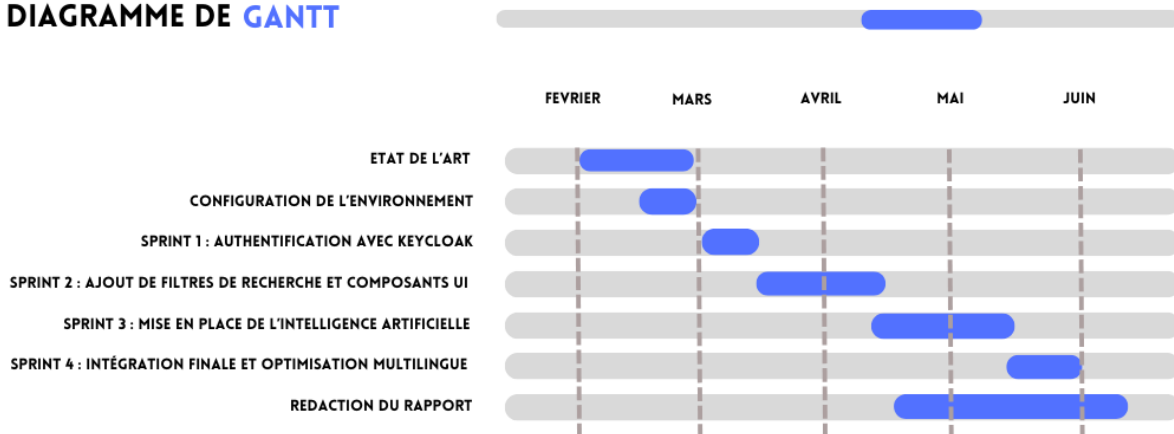
TABLE 1.3 – Backlog produit

Titres	Objectifs	Durée
Sprint 0	- État de l'art approfondi sur les technologies et concepts clés du projet. - Compréhension métier. - Configuration de l'environnement et exploration initiale.	Du 03-02-25 Au 28-02-25
Sprint 1	- Mise en place de l'authentification . - Consommation de l'API Search et affichage de la liste des dossiers.	Du 03-03-25 Au 14-03-25

Sprint 2	- Développement d'un moteur de recherche classique incluant une barre de recherche, des filtres dynamiques (dropdown, multiselect...). - Intégration de la pagination en lien direct avec les résultats fournis par l'API Search - Ajout d'une AppBar pour une navigation fluide et cohérente.	Du 17-03-23 Au 11-04-23
Sprint 3	- Étude comparative des modèles LLM et sélection du modèle performant. - Mise en place d'un système de prompt engineering adapté aux cas d'usage du projet. - Réglage et optimisation fine des paramètres du modèle pour un meilleur rendement.	Du 14-04-25 Au 09-05-25
Sprint 4	- Développement de l'API intelligente "ai-Search" en Spring Boot. - Fusion entre la recherche classique et la recherche basée sur l'intelligence artificielle. - Intégration de la gestion multilingue via l'implémentation de i18n . - Développement de la fonctionnalité de basculement entre organisations.	Du 12-05-25 Au 05-06-25

1.4.3 Diagramme de Gantt

La figure 1.6 montre un diagramme de Gantt [10] illustrant le déroulement des sprints tout au long de la période du stage.

DIAGRAMME DE GANTT**FIGURE 1.6 – Diagramme de Gantt de notre projet**

1.5 Conclusion

Dans ce chapitre, nous avons présenté l'entreprise Platana, ses solutions SAV, ainsi que la problématique liée à la gestion des dossiers et les limites de la solution actuelle. Une approche innovante, basée sur une recherche intelligente par IA, a été proposée, et la méthodologie Scrum a été choisie pour structurer le développement de notre projet. Le chapitre suivant détaille la spécification des besoins et les choix techniques pour la réalisation du projet.

Spécification des besoins et choix techniques

Sommaire

2.1	Introduction	17
2.2	Spécification des besoins	17
2.2.1	Identification des acteurs	17
2.2.2	Identification des besoins fonctionnels	18
2.2.3	Diagramme de cas d'utilisation global	18
2.2.4	Identification des besoins non fonctionnels	19
2.3	Besoins techniques	20
2.3.1	Environnement matériel	20
2.3.2	Environnement logiciel et technologies utilisées	21
2.3.3	Framework utilisées	22
2.3.4	Outils de gestion de projets et de communication	23
2.4	Approches de développement	24
2.4.1	Approches pour le développement de l'interface utilisateur	24
2.4.2	Approche de recherche basée sur l'intelligence artificielle	26
2.4.3	Internationalisation i18n	29
2.5	Conclusion	30

2.1 Introduction

Dans ce chapitre, nous détaillons les exigences ainsi que les fondations techniques nécessaires à la conception de notre application. Nous commençons par identifier les acteurs impliqués dans le système et leurs rôles respectifs, avant de préciser les besoins fonctionnels et non fonctionnels qui orientent le développement de l'application. Nous abordons ensuite les besoins techniques en présentant l'environnement matériel et logiciel utilisé, ainsi que les technologies et frameworks retenus. Enfin, nous introduisons les concepts fondamentaux, en comparant leur pertinence. Ce chapitre constitue une étape cruciale pour structurer le développement et garantir que la solution réponde aux attentes des utilisateurs tout en respectant les contraintes techniques.

2.2 Spécification des besoins

Nous entamons cette section par une analyse des différents acteurs intervenant dans le système, puis nous précisons les besoins fonctionnels et non fonctionnels associés.

2.2.1 Identification des acteurs

Le système cible trois types d'acteurs principaux, chacun ayant des rôles et des interactions spécifiques avec la plateforme .

- **Viewer** : Utilisateur en magasin, chargé de la recherche et de la gestion des dossiers SAV via l'interface. Ses responsabilités incluent :

- Effectuer des recherches de dossiers SAV.
- Consulter la liste et les détails des dossiers SAV.

- **Administrateur** : Administrateur du magasin, il peut remplir les rôles du viewer et dispose, en plus, d'une responsabilité supplémentaire :

- Créer des dossiers SAV.

- **Super administrateur** : Administrateur de la plateforme, il cumule les rôles de Viewer et Administrateur et bénéficie de privilèges supplémentaires, notamment :

- Basculer entre les organisations pour superviser et gérer les dossiers SAV de différents magasins ou entités.

2.2.2 Identification des besoins fonctionnels

Un projet a pour objectif principal de répondre à un besoin. Il est donc essentiel de formuler ce besoin de manière claire et précise avant d'envisager toute solution. En général, les besoins sont exprimés sous forme d'exigences fonctionnelles, qui décrivent ce que le produit ou le service issu du projet doit accomplir. Dans notre cas, le besoin fondamental auquel la solution doit répondre est la mise en place d'une interface de recherche à la fois intuitive et performante pour les dossiers du service après-vente, dans le but d'améliorer l'efficacité des opérateurs tout en satisfaisant les attentes d'une clientèle internationale.

Pour répondre à ce besoin, la solution envisagée devra intégrer les fonctionnalités suivantes :

- Authentification sécurisée.
- Recherche par filtres.
- Recherche par saisie textuelle.
- Affichage des dossiers SAV dans une grille interactive.
- Recherche intelligente et multilingue.
- Combinaison recherche classique et intelligente.

2.2.3 Diagramme de cas d'utilisation global

Le diagramme de cas d'utilisation, illustré dans la figure 2.8, présente une vue d'ensemble des interactions entre les différents acteurs et le système, mettant en évidence les fonctionnalités principales conçues pour répondre aux besoins spécifiques des utilisateurs dans le cadre de la gestion des dossiers SAV. Ce diagramme identifie les trois acteurs principaux : le Viewer, l'Admin, et le Super Admin, chacun doté de privilèges et d'actions distincts.

Le Viewer cet acteur peut consulter la liste des dossiers SAV et effectuer des recherches textuelles simples ("Rechercher dossiers par saisie textuelle") ou utiliser des filtres avancés

("Rechercher dossiers par filtres avancés"). Il dispose également de la possibilité d'effectuer une recherche intelligente, enrichissant ainsi ses options de navigation. De plus, le Viewer peut changer la langue de l'interface ("Changer la langue de l'interface"), une fonctionnalité qui soutient une expérience utilisateur multilingue. L'Admin cet acteur étend les capacités du Viewer et peut accéder à la page de création des dossiers ("Accéder à la page de création des dossiers"), ce qui lui permet de gérer activement les données SAV. Cette action inclut la création de nouveaux dossiers, renforçant la flexibilité de la gestion. Le Super Admin oté de privilèges élargis, le Super Admin étend les capacités de l'Admin et peut basculer entre les organisations ("Basculer entre les organisations"). Il a également accès aux dossiers SAV de toutes les organisations, lui conférant un contrôle global sur la configuration et la gestion du système.

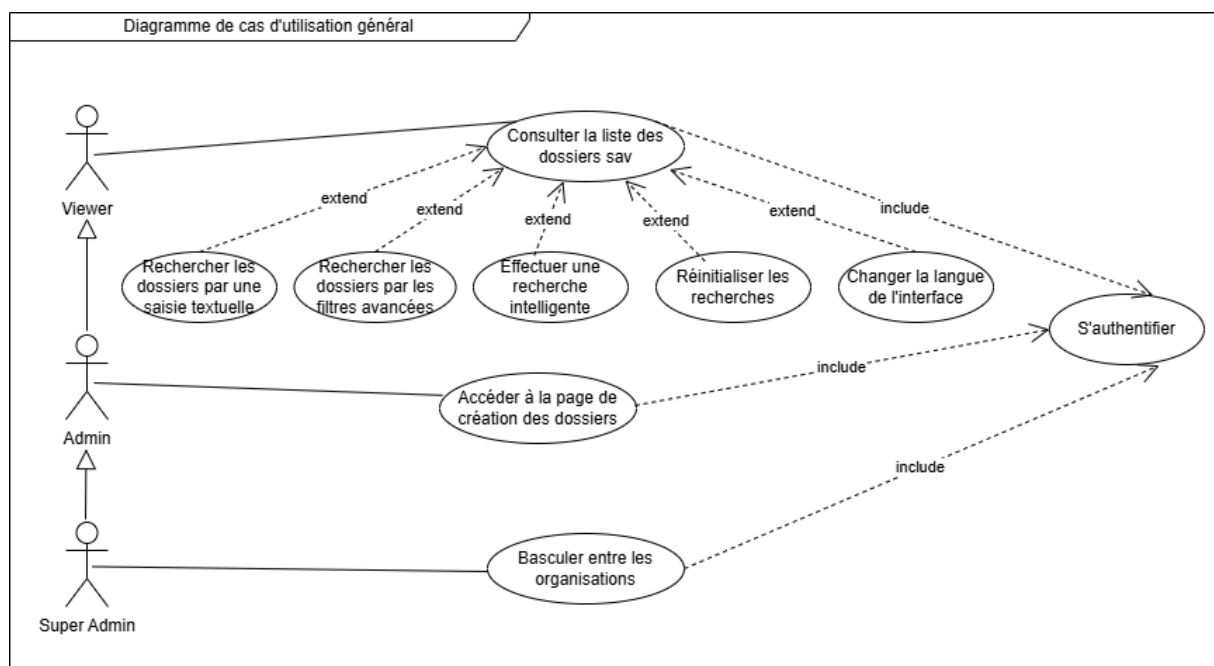


FIGURE 2.1 – Diagramme de cas d'utilisations général

2.2.4 Identification des besoins non fonctionnels

En complément des besoins fonctionnels mentionnés, notre application devra également satisfaire plusieurs besoins non fonctionnels visant à améliorer l'expérience utilisateur, tels que :

- Multilinguisme : L'interface et la recherche IA doivent supporter plusieurs langues (français, anglais, allemand, etc.) pour répondre à une clientèle internationale.

- Performance : Les temps de réponse doivent être optimisés, tant pour la recherche classique que pour la recherche assistée par IA, même en cas de forte charge.
- Sécurité : Les appels API doivent être sécurisés via Keycloak, avec une authentification basée sur des tokens pour protéger les données sensibles.
- Ergonomie : L'interface doit être intuitive, avec un design moderne et responsive, minimisant le temps d'apprentissage des utilisateurs.
- Compatibilité : La solution doit s'intégrer aux API existantes de Platana sans nécessiter de refonte du backend.
- Maintenabilité : Le code doit être modulaire et bien documenté pour faciliter les évolutions futures.

2.3 Besoins techniques

Les besoins techniques englobent tout ce qui garantit le bon fonctionnement de l'application, notamment l'environnement de travail. Le développement de notre solution mobilise divers outils, matériels et logiciels pour répondre aux exigences du projet.

2.3.1 Environnement matériel

Pour assurer le bon déroulement du développement, nous avons travaillé sur un poste équipé des caractéristiques techniques présentées dans le tableau 2.1 ci-dessous.

TABLE 2.1 – Environnement matériel utilisé

Caractéristiques	Ordinateur
Marque	Lenovo
Processeur	AMD Ryzen 5
Carte graphique	NVIDIA GeForce RTX 3050
Mémoire vive	16,0 Go
Disque dur	512 Go SSD
Système d'exploitation	Windows 11 Professionnel 64 bits

2.3.2 Environnement logiciel et technologies utilisées

Dans cette section, nous détaillons les logiciels ainsi que les langages de programmation employés pour le développement de notre plateforme.

— IntelliJ IDEA

IntelliJ IDEA [11], développé par JetBrains, est un environnement de développement intégré (IDE) multiplateforme destiné aux langages de la JVM. Son objectif est d'optimiser la productivité des développeurs en automatisant les tâches répétitives et fastidieuses. Il offre des fonctionnalités avancées telles que la complétion intelligente du code, l'analyse statique et les refactorisations, permettant ainsi aux utilisateurs de se concentrer sur les aspects créatifs et intéressants du développement logiciel.



FIGURE 2.2 – Logo IntelliJ IDEA

— Postman

Postman [12], une extension open source pour Chrome, est un puissant outil de débogage des requêtes HTTP. Il offre la possibilité d'envoyer différents types de requêtes telles que POST, PUT, GET, HEAD, avec des paramètres ou des en-têtes personnalisés. Il prend en charge plusieurs méthodes d'authentification, notamment Basic, OAuth et Digest. Les réponses reçues sont affichées avec une coloration syntaxique adaptée pour simplifier leur analyse.



FIGURE 2.3 – Logo Postman

— TypeScript

TypeScript [13] est un langage de programmation avancé, compilé, à la fois fortement typé et orienté objet. Il s'inscrit comme un environnement global, autant un langage qu'un écosystème

d'outils performants destinés à améliorer le processus de développement. Basé sur JavaScript, TypeScript fonctionne comme un sur-ensemble typé, ce qui signifie qu'il comprend toutes les fonctionnalités de JavaScript tout en offrant de nombreuses améliorations pour renforcer la sécurité, la stabilité et la qualité du code.



FIGURE 2.4 – Logo TypeScript

— Keycloak

Keycloak [14] est une solution open source appartenant à la catégorie des outils de gestion des identités et des accès (IAM – Identity and Access Management). Son objectif principal est de simplifier la protection des applications, des équipements et des services au sein du système d'information. Là où les développeurs devaient autrefois implémenter eux-mêmes des mécanismes d'authentification complexes, Keycloak fournit désormais ces fonctionnalités de manière native.



FIGURE 2.5 – Logo Keycloak

2.3.3 Framework utilisées

Ces frameworks sont sélectionnés pour leurs capacités à répondre aux exigences techniques et fonctionnelles de l'application SAV de Platana, en combinant robustesse, modularité et facilité d'intégration.

— Spring Boot

Spring Boot [15] est un framework open source en Java dédié au développement web, principalement axé sur l'architecture microservices. Il facilite la création d'environnements de production prêts à l'emploi en proposant une configuration initiale intégrée, permettant aux développeurs de démarrer rapidement sans avoir à gérer de nombreux réglages complexes.



FIGURE 2.6 – Logo Spring boot

— Angular

Angular [2], un framework open source développé et maintenu par Google, est conçu pour le développement d'applications web côté client. Dans sa version 18, Angular continue d'évoluer en intégrant des optimisations de performance, une meilleure gestion de la modularité ainsi que des outils modernes pour les développeurs. Il repose sur TypeScript, une surcouche typée de JavaScript, permettant d'écrire du code plus robuste et maintenable.



FIGURE 2.7 – Logo Angular

— Draw.io

Draw.io [16] est une plateforme en ligne gratuite permettant de réaliser des diagrammes et des illustrations techniques. Son interface intuitive facilite la conception de divers types de diagrammes, tels que les organigrammes, les diagrammes de flux ou UML. Elle offre également des fonctionnalités de collaboration en temps réel et de partage des diagrammes via le cloud, favorisant le travail d'équipe et la simplicité d'accès.



FIGURE 2.8 – Logo Draw.io

2.3.4 Outils de gestion de projets et de communication

Pour optimiser la gestion de projet et faciliter la collaboration entre les membres de l'équipe, GitLab est l'un des outils majeurs utilisés.

— GitLab

GitLab [17] est une plateforme open source de développement logiciel tout-en-un, conçue pour répondre aux besoins modernes des équipes de développement. Elle intègre de manière transparente un système de gestion de version avancé, un suivi détaillé des problèmes, un mécanisme de revue de code collaboratif, ainsi que des outils puissants pour l'automatisation via CI/CD. La plateforme favorise également une collaboration efficace entre les membres de l'équipe grâce à ses fonctionnalités de partage, de revue et de communication intégrée, renforçant ainsi la cohérence et la qualité globale du logiciel développé.



FIGURE 2.9 – Logo GitLab

2.4 Approches de développement

Cette section introduit les concepts et technologies fondamentaux utilisés dans le projet, à savoir Syncfusion, les Large Language Models (LLM) [8] et i18n [18], afin de poser les bases de la solution proposée.

2.4.1 Approches pour le développement de l'interface utilisateur

Dans le cadre de la refonte de l'interface utilisateur, deux solutions principales ont été envisagées pour répondre aux besoins d'amélioration de l'application SAV de Platana.

2.4.1.1 Étude de l'approche Material-UI

Material-UI [7], est une bibliothèque open source qui implémente le Material Design de Google. Elle propose une collection complète de composants préconçus, prêts à être utilisés en production dès leur installation.



FIGURE 2.10 – Logo Material UI

- Limitations fonctionnelles : Les composants de base (boutons, formulaires, grilles) manquent d’interactivité native pour des scénarios métiers complexes, nécessitant souvent des ajustements manuels pour s’adapter aux besoins spécifiques des opérateurs.
- Fragmentation technique : L’usage de bibliothèques multiples entraînait des conflits de versions, une surcharge de styles et un manque de cohérence visuelle.

2.4.1.2 Approche basée sur Syncfusion

Syncfusion [3], est une bibliothèque d’interface utilisateur (UI) complète, offrant une large gamme de composants préconçus tels que des grilles, des barres de recherche, des filtres et des outils de pagination. Elle se positionne comme une solution tout-en-un pour les applications professionnelles, alliant richesse fonctionnelle et flexibilité en matière de design.



FIGURE 2.11 – Logo Syncfusion

- Design adaptatif : Des thèmes prédéfinis (Material, Fluent, Tailwind) et une personnalisation via CSS/Sass permettent de créer des interfaces métier sans sacrifier l’esthétique.
- Interactivité native : Des composants conçus pour l’efficacité : grilles de données avec tri/filtrage dynamique, calendriers de gestion de tâches, et diagrammes interactifs.
- Cohérence technique : Une bibliothèque unifiée éliminant les risques d’incompatibilités, avec une expérience utilisateur homogène sur tous les modules.

2.4.1.3 Synthèse comparative et approche retenue

Pour mieux illustrer les différences entre ces deux approches, le tableau 2.2 propose une synthèse comparative basée sur plusieurs critères clés :

- Cohérence technique : Degré d'uniformité et de compatibilité au sein de la bibliothèque, permettant de réduire les risques de conflits ou de surcharge stylistique.
- Intégration facile avec Angular : Simplicité et efficacité de l'intégration au sein d'un environnement Angular.
- Support dédié : Disponibilité d'un support technique ou de ressources pour accompagner la résolution des problèmes et l'optimisation.
- Ergonomie intuitive : Capacité de l'interface à offrir une expérience utilisateur fluide, adaptée aux besoins métier et facile à naviguer.

TABLE 2.2 – Comparaison des fonctionnalités de Material-UI et Syncfusion

Critère	Material-UI	Syncfusion
Cohérence technique	✗	✓
Intégration facile avec Angular	✓	✓
Support dédié	✗	✓
Ergonomie intuitive	✗	✓

À l'issue de cette analyse comparative, le choix s'est porté sur Syncfusion [3] en raison de sa cohérence technique élevée grâce à une bibliothèque unifiée, de son intégration facile avec Angular, de son support dédié professionnel, et de son ergonomie intuitive adaptée aux besoins métier complexes de l'application SAV de Platana.

2.4.2 Approche de recherche basée sur l'intelligence artificielle

Dans le cadre de ce projet, deux approches issues du traitement automatique du langage ont été explorées afin de permettre une recherche intelligente : les modèles LLM [8] (Large Language Models) et la reconnaissance d'entités nommées NER [18] (Named Entity Recognition). Cette section présente une étude comparative des deux approches et justifie le choix de la solution retenue

2.4.2.1 Étude de l'approche basée sur les LLM

Les LLM [8] (Large Language Models) sont des modèles de langage entraînés sur d'énormes volumes de données textuelles multilingues. Leur principal avantage est leur capacité à comprendre la sémantique du langage naturel, à généraliser à partir d'exemples, et à générer des sorties structurées à partir de requêtes libres, sans besoin d'entraînement spécifique pour chaque tâche.

Large Language Models (LLMs)

aicorr.com

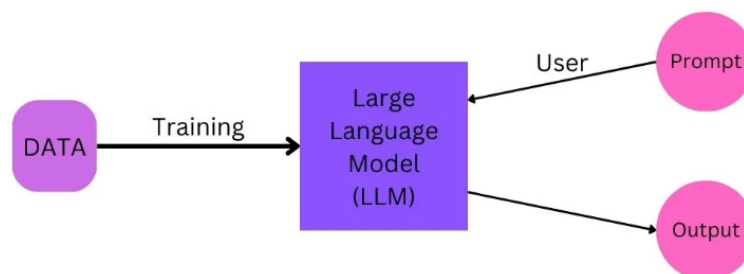


FIGURE 2.12 – Large Language Models

Les avantages clés de cette approche :

- Support multilingue natif : Comprend plusieurs langues sans configuration ou modèle distinct.
- Compréhension sémantique avancée : Peut reconnaître des synonymes ou reformulations et les mapper au bon filtre.
- Intégration simple : il suffit d'envoyer une requête à l'API pour obtenir une réponse structurée avec un effort de développement minimal.
- Évolutivité naturelle : un seul modèle permet de prendre en charge de nouvelles langues, reformulations ou filtres sans nécessiter de réentraînement.

2.4.2.2 Étude de l'approche NER

La NER [19] (Reconnaissance d'Entités Nommées) est une approche classique du NLP (traitement automatique du langage), consistant à détecter et classer automatiquement des entités clés dans un texte. Elle repose souvent sur des bibliothèques comme spaCy ou des modèles préentraînés Hugging Face. La figure 2.13 illustre un exemple d'extraction d'entités à partir d'un texte.



FIGURE 2.13 – Extraction d'entités à partir d'un texte

Bien que couramment utilisée dans des applications NLP traditionnelles, cette approche a montré plusieurs limitations dans notre contexte.

- Support multilingue limité : nécessite un modèle distinct pour chaque langue, par exemple `fr_core_news_md` pour le français et `en_core_web_md` pour l'anglais.
- Performance inégale selon la langue : Les résultats sont généralement meilleurs en anglais que dans d'autres langues comme l'arabe ou le polonais.
- Intégration complexe : Contrairement aux LLM [8], l'intégration des modèles NER nécessite davantage de configuration et d'adaptation selon les langues et cas d'usage.
- Nécessité d'annotation manuelle : Chaque langue requiert des centaines, voire des milliers, d'exemples annotés pour obtenir des performances satisfaisantes, ce qui implique un coût élevé en temps et en ressources.

2.4.2.3 Synthèse comparative et approche retenue

Afin de mieux visualiser les différences entre ces deux approches, le tableau 2.3 présente une synthèse comparative selon des critères essentiels tels que :

- Multilinguisme : capacité du système à prendre en charge et à traiter plusieurs langues efficacement .
- Intégration : facilité avec laquelle le système peut être intégré dans une application ou un environnement existant
- Évolutivité : potentiel du système à s’adapter ou à évoluer face à de nouvelles exigences ou à une augmentation du volume de données .
- Performance : niveau de stabilité, de rapidité et de fiabilité des résultats produits par le système.

TABLE 2.3 – Comparaison entre LLM et NER

Critère	LLM	NER
Multilinguisme	Natif (plusieurs langues)	Limité (modèles séparés)
Intégration	Simple (via API)	Complexe (configuration requise)
Évolutivité	Facile (sans réentraînement)	Difficile (annotation requise)
Performance	Stable	Variable selon la langue

À l’issue de cette analyse comparative, le choix s’est porté sur les modèles de langage de grande échelle (LLM) en raison de leur capacité native à gérer le multilinguisme, de leur intégration simplifiée via API, de leur évolutivité sans réentraînement, et de leur performance stable, répondant ainsi aux exigences de la recherche intelligente dans l’application SAV de Platana.

2.4.3 Internationalisation i18n

i18n [18], ou internationalization, est le processus de conception d’une application afin qu’elle puisse être facilement adaptée à différentes langues et régions sans avoir besoin de modifications techniques majeures.

Dans ce projet, il n'est pas permis de :

- Gérer dynamiquement plusieurs langues dans l'interface utilisateur.
- Proposer une expérience multilingue.
- Faciliter l'adoption de l'application dans un contexte international.

2.5 Conclusion

Ce chapitre a défini les besoins fonctionnels et non fonctionnels du module de recherche intelligente, en précisant les attentes des opérateurs et les limites de la solution existante. Le choix d'Angular 18 et de Syncfusion a été justifié pour améliorer l'ergonomie et la performance. Une comparaison avec Material-UI a mis en évidence les avantages de Syncfusion pour répondre aux besoins métiers. Enfin, l'analyse architecturale a posé les bases du développement. Le chapitre suivant est dédié à la mise en place des fondations fonctionnelles de la solution.

Mise en place des fondations fonctionnelles

Sommaire

3.1	Introduction	32
3.2	Sprint 1 : Mise en œuvre de l'authentification et affichage des données	32
3.2.1	Backlog du sprint 1	32
3.2.2	Diagramme de cas d'utilisation du sprint 1	34
3.2.3	Diagrammes de séquence du sprint 1	37
3.2.4	Diagramme de classes du sprint 1	39
3.2.5	Réalisation	39
3.3	Sprint 2 : Ajout de filtres de recherche et composants UI . .	41
3.3.1	Backlog du sprint 2	41
3.3.2	Diagramme de cas d'utilisation du sprint 2	41
3.3.3	Diagrammes de séquence du sprint 2	45
3.3.4	Diagramme de classes du sprint 2	48
3.3.5	Réalisation	48
3.4	Conclusion	51

3.1 Introduction

Ce chapitre présente les avancées réalisées lors des sprints 1 et 2, qui ont posé les bases fonctionnelles de l'application SAV de Platana. Le sprint 1 a été consacré à l'implémentation d'une authentification sécurisée, intégrant un mécanisme de gestion des tokens et un affichage optimisé des données des dossiers SAV grâce à la grille Syncfusion. Le sprint 2 a enrichi cette fondation en développant des fonctionnalités de recherche, de navigation et de personnalisation de l'interface, renforçant ainsi l'ergonomie et la flexibilité pour les utilisateurs.

3.2 Sprint 1 : Mise en œuvre de l'authentification et affichage des données

Le sprint 1 est dédié à la mise en place des fondations techniques de notre projet. L'objectif principal est de sécuriser l'accès à l'interface et aux API, tout en permettant un affichage efficace des données des dossiers SAV pour les utilisateurs.

3.2.1 Backlog du sprint 1

Les tâches assignées pour ce sprint sont détaillées dans le tableau 3.1.

TABLE 3.1 – Backlog du sprint 1

Nom du sprint	Tâches
Mise en œuvre de l'authentification et affichage des données	Création d'un fichier de configuration pour définir le realm et le client ID, facilitant la gestion des environnements.
	Développement et implémentation du service d'authentification Angular avec flux PKCE.
	Développement de l'intercepteur HTTP pour injection automatique des tokens.
	Mise en place du mécanisme de rafraîchissement automatique des tokens.

	Développement d'un service pour consommer l'API search avec intégration du token Keycloak dans l'en-tête.
	Affichage de la liste des dossiers SAV dans une grille Syncfusion

Dans le cadre du Sprint 1, notre objectif principal était de mettre en place une authentification sécurisée et un affichage efficace des données des dossiers SAV pour les opérateurs. Pour ce faire, nous avons intégré Keycloak pour assurer une gestion robuste des identités et des accès, combinée à l'utilisation de la grille Syncfusion pour une présentation claire et optimisée des données SAV dans l'application Angular. Cette architecture favorise une interaction sécurisée et intuitive entre l'opérateur, l'application, et les API de Platana, tout en intégrant la validation des tokens comme élément central du processus.

La figure 3.1 suivante présente une représentation détaillée du processus d'authentification et de validation des tokens avec Keycloak, mettant en évidence les étapes clés telles que la redirection vers l'application, la vérification des identifiants, la validation ou l'invalidation des tokens, et l'appel sécurisé à l'API pour afficher les données des dossiers SAV via Syncfusion.

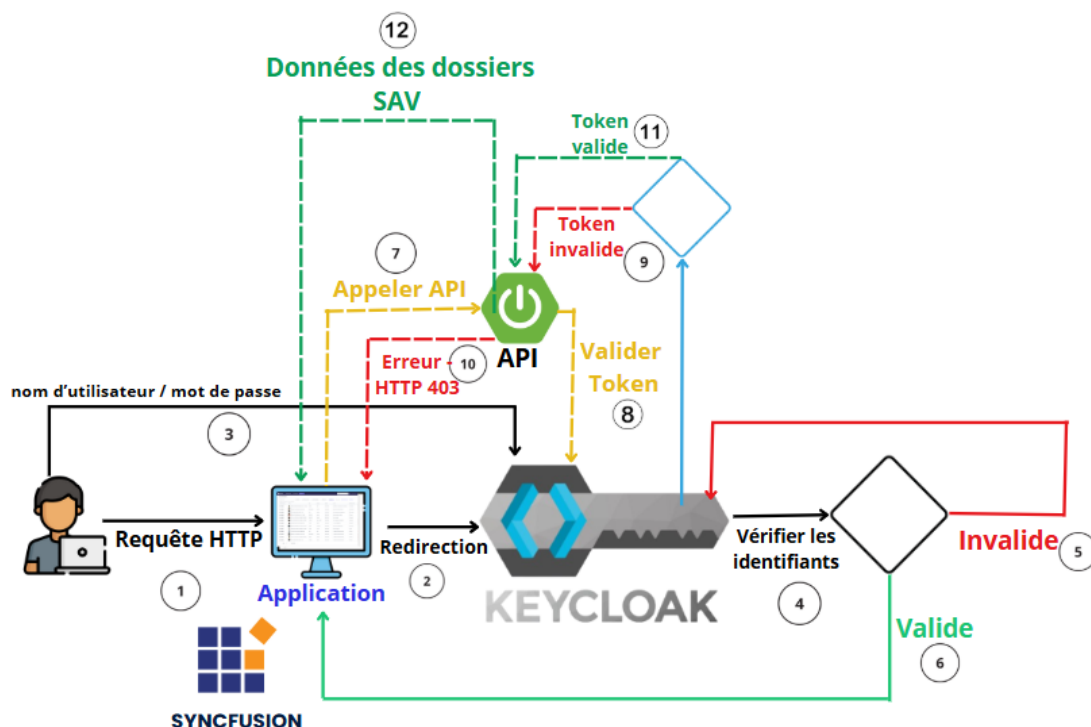


FIGURE 3.1 – Processus d'authentification et affichage des dossiers SAV

3.2.2 Diagramme de cas d'utilisation du sprint 1

Le diagramme de cas d'utilisation du Sprint 1, présenté à la figure 3.2, illustre les interactions entre les acteurs et le système lors de la phase initiale du projet. L'acteur, après authentification, peut accéder au cas d'utilisation principal «Afficher la liste des dossiers SAV» pour visualiser les données SAV via une interface dédiée.

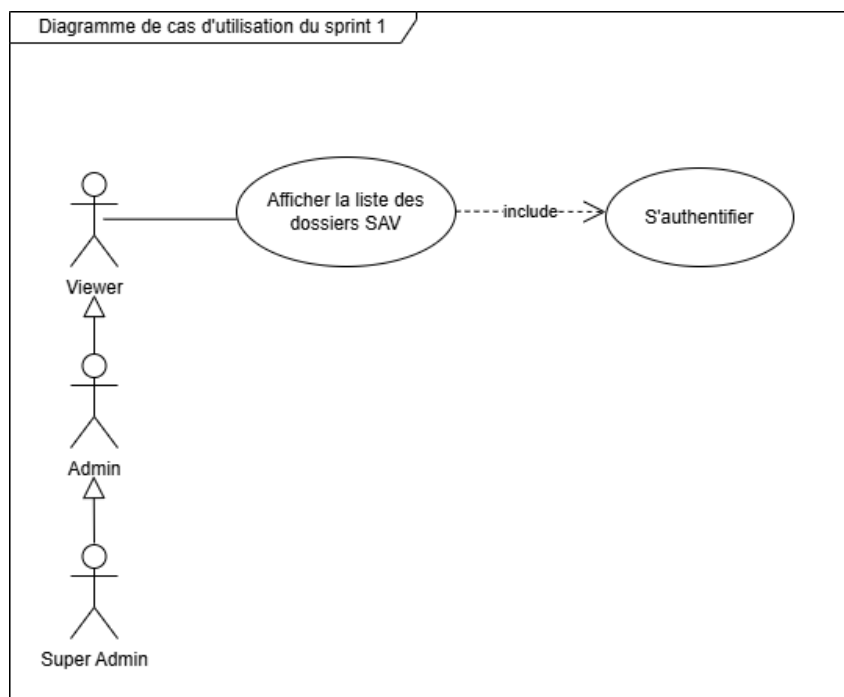


FIGURE 3.2 – Diagramme du cas d'utilisation du sprint 1

Le tableau 3.2 présente la description textuelle du cas d'utilisation "S'authentifier", qui permet aux différents acteurs de se connecter et d'obtenir un token pour sécuriser les appels API.

TABLE 3.2 – Description textuelle du cas d'utilisation « S'authentifier »

Titre	S'authentifier
Acteur	Super admin, Admin, Viewer
Objectif	L'acteur doit s'authentifier pour accéder à l'interface et obtenir un Bearer Token afin de sécuriser les appels API Platana.

Pré-condition	L'acteur dispose d'un compte approuvé dans Keycloak.
Post-condition	L'acteur est authentifié, un Bearer Token est obtenu, et les appels API sont sécurisés.
Scénario nominal	1. L'acteur accède à l'interface Angular et est redirigé vers la page de connexion Keycloak. 2. L'acteur saisit son email et son mot de passe dans l'interface de Keycloak. 3. L'acteur clique sur le bouton "Connexion". 4. Keycloak vérifie les identifiants et génère un token JWT si les identifiants sont valides. 5. Keycloak redirige L'acteur vers l'interface Angular avec le token. 6. L'interface Angular stocke le token dans le local storage ou en mémoire. 7. Lors d'un appel à une API Platana, Angular inclut le token dans l'en-tête. 8. L'API Platana valide le token via Keycloak et, si valide, retourne les données.
Scénario alternatif	2.1. Si les champs email ou mot de passe sont vides, Keycloak affiche un message d'erreur : "Champs obligatoires vides". 4.1. Si l'email est invalide, Keycloak affiche un message d'erreur : "Email invalide". 4.2. Si le mot de passe est incorrect, Keycloak affiche un message d'erreur : "Mot de passe invalide". 8.1. Si le token est invalide ou expiré, l'API Platana retourne une erreur HTTP 401, et Angular redirige l'acteur vers Keycloak pour une nouvelle authentification.

Le tableau 3.3 présente la description textuelle du cas d'utilisation « Afficher la liste des dossiers SAV ».

TABLE 3.3 – Description textuelle du cas d'utilisation « Afficher la liste des dossiers SAV »

Titre	Afficher la liste des dossiers SAV
Acteur	Super admin, Admin, Viewer
Objectif	Permettre à l'acteur d'afficher la liste des dossiers SAV.
Pré-condition	L'acteur doit être authentifié et disposer des droits nécessaires pour consulter les dossiers SAV.
Post-condition	La liste des dossiers SAV est affichée dans l'interface..
Scénario nominal	1. L'acteur accède à l'interface Angular et est redirigé vers la page de connexion de Keycloak. 2. L'acteur saisit son email et son mot de passe dans l'interface de Keycloak. 3. L'acteur clique sur le bouton "Connexion". 4. Keycloak vérifie les identifiants et génère un token JWT si les identifiants sont valides. 5. Keycloak redirige l'acteur vers l'interface Angular avec le token. 6. Le système affiche la liste des dossiers SAV.
Scénario alternatif	2.1. Si les champs email ou mot de passe sont vides, Keycloak affiche un message d'erreur : "Champs obligatoires vides". 4.1. Si l'email est invalide, Keycloak affiche un message d'erreur : "Email invalide". 4.2. Si le mot de passe est incorrect, Keycloak affiche un message d'erreur : "Mot de passe invalide".

3.2.3 Diagrammes de séquence du sprint 1

Le diagramme de séquence présenté dans la figure 3.3 décrit le processus d'authentification via Keycloak pour le cas d'utilisation « S'authentifier ». Il illustre les échanges entre l'utilisateur, l'application, Keycloak et l'API. Après la saisie des identifiants, un token est délivré en cas de succès, permettant un accès sécurisé à l'API. En cas d'échec, une erreur 403 est retournée. Le diagramme met en évidence les étapes clés de validation et de sécurisation de l'accès.

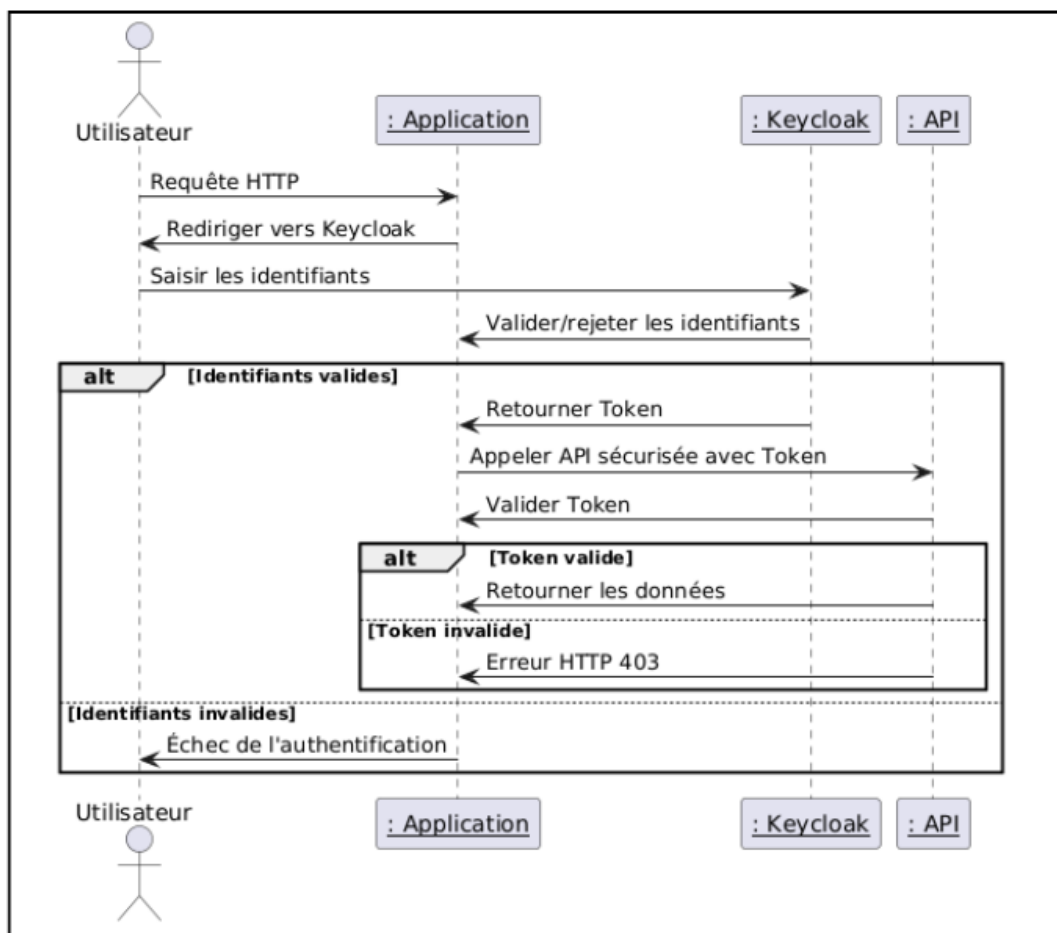


FIGURE 3.3 – Diagramme de séquence du cas d'utilisation « S'authentifier »

La figure 3.4 présente le diagramme de séquence du cas d'utilisation « Afficher la liste des dossiers SAV ». Elle illustre les interactions entre l'utilisateur, l'interface, les services et l'API. Une requête déclenchée par l'utilisateur transite par les services de configuration, puis est envoyée via HttpClient à l'API, qui retourne les données pour être affichées dans l'interface.

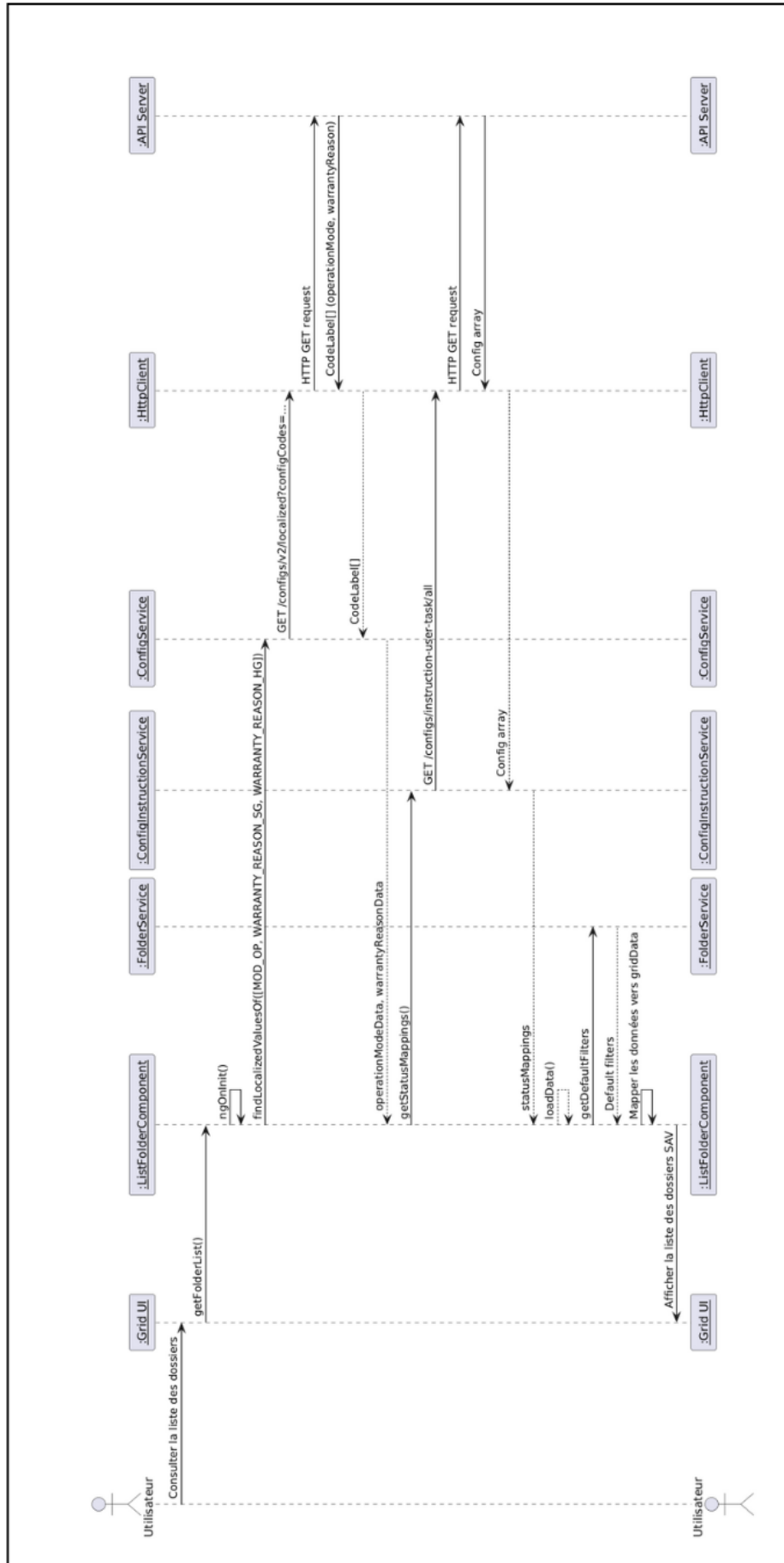


FIGURE 3.4 – Diagramme de séquence du cas d'utilisation « Afficher la liste des dossiers SAV »

3.2.4 Diagramme de classes du sprint 1

Le diagramme de classes correspondant au premier sprint, illustré dans la Figure 3.5, présente la structure des entités principales développées pour la gestion des dossiers SAV. Il met en évidence les relations entre les classes Viewer, Role, Organization, Dossiers SAV, et Image produit. Le Viewer est associé à un Role, qui inclut la méthode S'authentifier. Le Viewer est également lié à une Organization. La classe Dossiers SAV contient des attributs tels que incrementalNumber, productCode, et warrantyReason,... et est associée à la classe Image produit via la méthode getImageUrl.

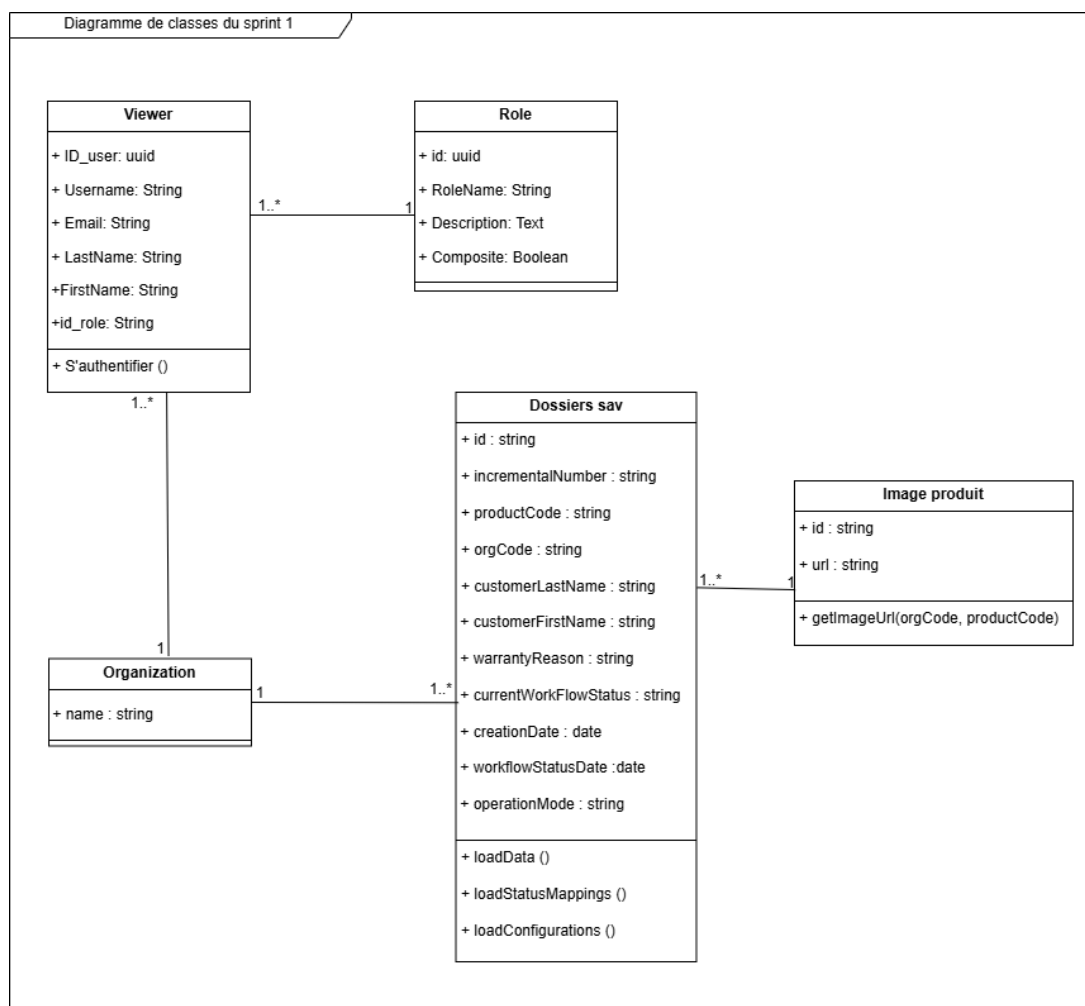


FIGURE 3.5 – Diagramme de classes du sprint 1

3.2.5 Réalisation

Cette section présente les interfaces intégrées et développées au cours du premier sprint, dans le but de clarifier les tâches réalisées.

La figure 3.6 illustre l’interface d’authentification fournie par Keycloak, intégrée à notre projet grâce à une configuration déjà détaillée dans la section précédente.

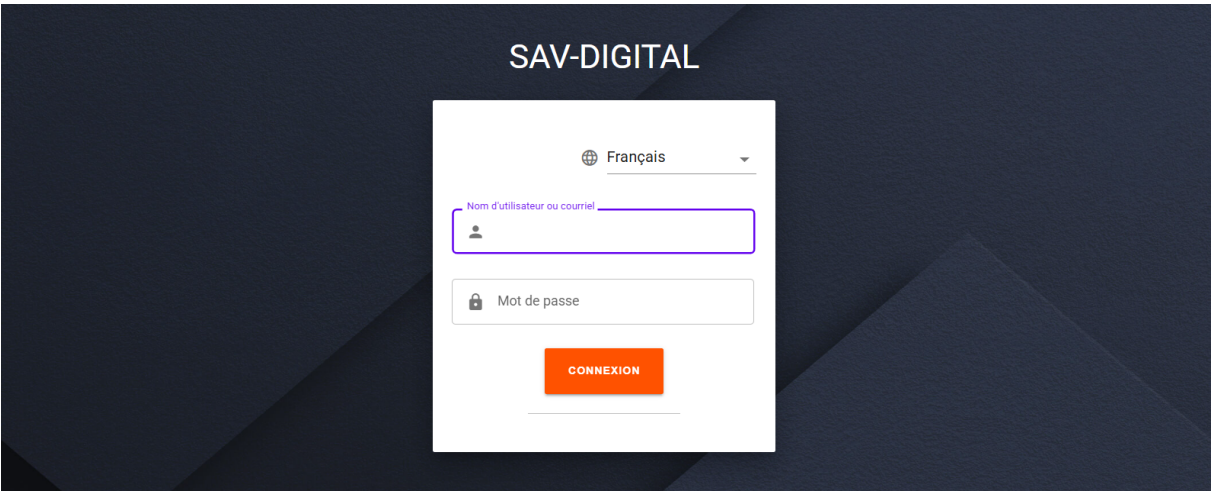


FIGURE 3.6 – Interface d’authentification avec keycloak

Lorsque l’utilisateur s’authentifie, les appels aux API Platana sont sécurisés, ce qui permet de consommer leurs services de manière protégée.

Ainsi, l’utilisateur authentifié reçoit un token JWT généré par Keycloak. Ce token est automatiquement inclus dans les en-têtes des requêtes, ce qui permet d’accéder aux API Platana.

Dans la figure 3.7, nous pouvons observer que les données des dossiers SAV, fournies par l’API /search, sont affichées dans une grille développée avec Syncfusion.

NUMÉRO DOSSIER	PRODUIT	NOM	PRÉNOM	CRÉATION	STATUT	DERNIÈRE ACTIVITÉ	RAISON DE GARANTIE	
8087	Vélo électrique de ville WAYSCRAL Hybrid ...	ZAGHDENE	YASMINE	15/04/2025	Accord du fournisseur dépassé	🕒 il y a 1 an	En période de garantie constructeur	
8086	Vélo électrique de ville WAYSCRAL Hybrid ...	ZAGHDENE	YASMINE	15/04/2025	Accord du fournisseur dépassé	🕒 il y a 1 an	En période de garantie constructeur	
8072	Vélo électrique de ville WAYSCRAL Hybrid ...	AYEDI	AMAL	14/04/2025	Déclaration du transport	🕒 il y a 1 an	Non couvert par la garantie	
8039	pc			18/02/2025		🕒 il y a 3 ans	En période de garantie constructeur	
8027	Vélo électrique de ville WAYSCRAL Hybrid ...	ZAGHDENE	YASMINE	04/02/2025		🕒 il y a 3 ans	En période de garantie constructeur	
8026	Vélo électrique de ville WAYSCRAL Hybrid ...	ZAGHDENE	YASMINE	04/02/2025		🕒 il y a 3 ans	En période de garantie constructeur	
8001	Autoradio PIONEER SPH-DA160DAB Androl...	AYEDI	AMAL	23/12/2024		🕒 il y a 4 ans	En période de garantie constructeur	
8000	Autoradio PIONEER SPH-DA160DAB Androl...	AYEDI	AMAL	23/12/2024		🕒 il y a 4 ans	En période de garantie constructeur	
7999	Autoradio PIONEER SPH-DA160DAB Androl...	AYEDI	AMAL	23/12/2024		🕒 il y a 4 ans	En période de garantie constructeur	
7941	554593	A	A	21/11/2024		🕒 il y a 5 ans	Durée de garantie dépassée	
7940	554593	A	A	21/11/2024		🕒 il y a 5 ans	Durée de garantie dépassée	
7925	Vélo électrique de ville WAYSCRAL Hybrid ...			20/11/2024		🕒 il y a 5 ans	En période de garantie constructeur	
7924	Vélo électrique de ville WAYSCRAL Hybrid ...			20/11/2024		🕒 il y a 5 ans	En période de garantie constructeur	
7922	Vélo électrique de ville WAYSCRAL Hybrid ...			20/11/2024		🕒 il y a 5 ans	En période de garantie constructeur	
7921	Vélo électrique de ville WAYSCRAL Hybrid ...			20/11/2024		🕒 il y a 5 ans	En période de garantie constructeur	
7916	Vélo électrique de ville WAYSCRAL Hybrid ...			07/11/2024		🕒 il y a 5 ans	Durée de garantie dépassée	
7887	Vélo électrique de ville WAYSCRAL Hybrid ...			08/06/2024	Objet de réclamation	🕒 il y a 8 ans	Durée de garantie dépassée	

FIGURE 3.7 – Interface de la liste des dossiers SAV affichés dans une grid syncfusion

3.3 Sprint 2 : Ajout de filtres de recherche et composants UI

Cette section présente le backlog du sprint 2 ainsi que l'implémentation de la recherche classique et des composants UI , afin d'améliorer l'expérience utilisateur.

3.3.1 Backlog du sprint 2

Le tableau 3.4 représente le backlog du deuxième sprint.

TABLE 3.4 – Backlog du sprint 2

Nom du sprint	Tâches
Ajout de filtres de recherche et composants UI	Implémentation de la barre de recherche .
	Développement des filtres avancés (dropdowns, multiselect,...)
	Intégration des sélecteurs de date (création/modification).
	Connexion des filtres à l'API search.
	Implémentation de la pagination synchronisée avec l'API.
	Mise en place du système "Plus" pour les filtres additionnels.
	Création de l'AppBar avec fonctionnalités de navigation.
	Implémentation de la fonctionnalité de réinitialisation de la recherche

Dans le cadre du sprint 2, notre objectif est de mettre en place un système de recherche classique et des composants UI afin d'améliorer l'efficacité des utilisateurs SAV. Pour ce faire, nous adoptons une approche modulaire avec Angular 18 et Syncfusion, assurant une intégration fluide avec l'API search, tout en optimisant performances et ergonomie.

3.3.2 Diagramme de cas d'utilisation du sprint 2

Nous présentons ici la partie fonctionnelle développée lors du sprint 2 le diagramme de cas d'utilisation de la figure 3.8 donne une vue d'ensemble des fonctionnalités accessibles aux utilisateurs SAV et les principaux utilisateurs de l'application.

Quatre cas d'utilisation clés ont été identifiés, centrés sur la recherche et l'interaction avec l'interface : « Rechercher les dossiers SAV », « Naviguer dans les résultats », « Personnaliser l'interface », et « Accéder à la page de création des dossiers ». Ces fonctionnalités permettent aux utilisateurs de filtrer les dossiers, d'explorer les résultats obtenus et d'adapter l'interface à leurs besoins. Nous décrivons ci-après chacun de ces cas d'utilisation.

Le cas d'utilisation « Rechercher les dossiers SAV » permet à L'utilisateur d'effectuer des recherches à l'aide de la barre principale et de filtres avancés (menus déroulants, sélections multiples, sélecteurs de dates) .Il peut également enrichir sa recherche via le système de filtres supplémentaires ("Plus"). Ces actions déclenchent des appels à l'API '/search' pour obtenir les résultats correspondants. Le cas d'utilisation « Naviguer dans les résultats » offre à L'utilisateur la possibilité d'utiliser la pagination pour parcourir les pages de résultats, et d'ouvrir un dossier pour consulter ses détails. Le cas d'utilisation « Personnaliser l'interface » permet de configurer l'affichage des colonnes dans la grille Syncfusion (ajout ou suppression de colonnes). Enfin le cas « Accéder à la page de création des dossiers » : cette fonctionnalité permet de rediriger l'utilisateur vers une interface existante, déjà implémentée dans l'application Platana, dédiée à la création de dossiers. Ces fonctionnalités visent à garantir une recherche efficace ainsi qu'une expérience utilisateur fluide et adaptée aux préférences de l'utilisateur .

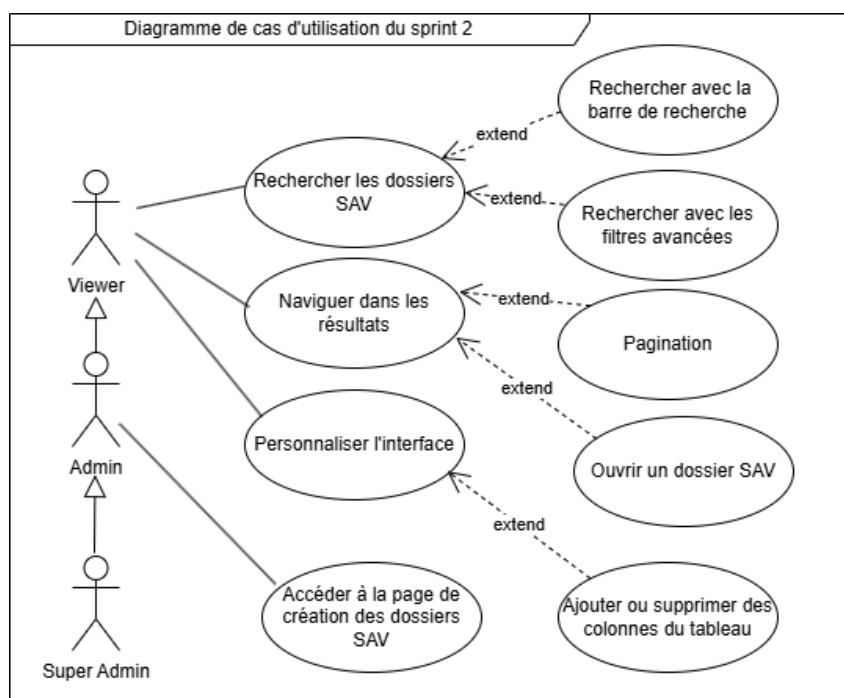


FIGURE 3.8 – Diagramme de cas d'utilisation du sprint 2

Le tableau 3.5 résume la description textuelle du cas d'utilisation « Rechercher les dossiers SAV ».

TABLE 3.5 – Description textuelle du cas d'utilisation « Rechercher les dossiers SAV »

Titre	Rechercher les dossiers SAV
Acteur	Super admin, Admin, Viewer
Résumé	L'acteur recherche les dossiers SAV en utilisant des filtres avancés et affiche les résultats dans une grille Syncfusion.
Pré-condition	L'acteur est authentifié via Keycloak et a accès à l'interface.
Post-condition	Les dossiers SAV correspondant aux critères de recherche sont affichés dans une grille paginée.
Scénario nominal	1. L'acteur accède à la section de recherche de l'interface. 2- L'acteur saisit un critère dans la barre de recherche. 3- L'acteur sélectionne des filtres via dropdowns ou multiselect. 4- L'acteur utilise les sélecteurs de date pour filtrer par date de création. 5- L'acteur clique sur le bouton "Plus" pour ajouter des filtres supplémentaires. 6-Le système envoie une requête à l'API /search avec les filtres appliqués. 7-Le système affiche les résultats dans une grille Syncfusion avec pagination.
Scénario alternatif	A1 : Filtres invalides 1- L'acteur sélectionne des filtres incompatibles par exemple date de création future. 2- le grid affiche pas de résultat à afficher. 3- Le scénario reprend au point 3 du scénario nominal.

Le tableau 3.6 résume la description textuelle du cas d'utilisation « Naviguer dans les résultats ».

TABLE 3.6 – Description textuelle du cas d'utilisation « Naviguer dans les résultats »

Titre	Naviguer dans les résultats
Acteur	Super admin, Admin, Viewer
Résumé	L'acteur parcourt les résultats de recherche et consulte les détails d'un dossier.
Pré-condition	Une recherche a été effectuée et les résultats sont affichés dans la grille Syncfusion.
Post-condition	L'acteur a parcouru les résultats ou consulté les détails d'un dossier.
Scénario nominal	1- L'acteur visualise les résultats dans la grille Syncfusion. 2- L'acteur utilise la pagination pour passer à la page suivante. 3- Le système met à jour la grille avec les nouveaux résultats. 4- L'acteur clique sur un dossier pour consulter ses détails. 5- Le système affiche les détails du dossier dans une vue dédiée. 6- L'acteur retourne à la grille .
Scénario alternatif	A1 : Informations manquantes ou invalides 1- Le système détecte des informations manquantes ou invalides et affiche un message d'erreur indiquant les champs nécessitant une correction. 2- Le scénario reprend au point 2 du scénario nominal.

Le tableau 3.7 résume la description textuelle du cas d'utilisation «Personnaliser l'interface».

TABLE 3.7 – Description textuelle du cas d'utilisation « Personnaliser l'interface »

Titre	Personnaliser l'interface
Acteur	Super admin, Admin, Viewer
Résumé	L'acteur configure les colonnes de la grille Syncfusion.
Pré-condition	L'acteur est authentifié via Keycloak et a accès à l'interface.
Post-condition	les colonnes de la grille sont mises à jour.

Scénario nominal	1- L'acteur accède à l'interface principale. 2- L'acteur clique sur l'icône de configuration de la grille Syncfusion. 3- L'acteur ajoute ou supprime des colonnes. 4- Le système met à jour la grille selon les préférences. 5- Le système affiche un message de confirmation indiquant que les modifications ont été appliquées.
------------------	--

Le tableau 3.8 résume la description textuelle du cas d'utilisation « Accéder à la page de création des dossiers ».

TABLE 3.8 – Description textuelle du cas d'utilisation « Accéder à la page de création des dossiers »

Titre	Accéder à la page de création des dossiers
Acteur	Super admin, Admin
Résumé	L'acteur accède à la page de création des dossiers.
Pré-condition	L'acteur est authentifié via Keycloak et a accès à l'interface.
Post-condition	La page de création des dossiers est affichée.
Scénario nominal	1- L'acteur clique sur le bouton "Créer". 2- Le système affiche la page de création des dossiers. 3- L'acteur peut revenir à la page de la liste des dossiers SAV.

3.3.3 Diagrammes de séquence du sprint 2

La figure 3.9 présente le diagramme de séquence illustrant le cas d'utilisation « Rechercher les dossiers SAV ». Il détaille les interactions entre l'Utilisateur, l'Interface, l'API Platana, et Keycloak. Le processus commence par la sélection d'un filtre par l'utilisateur, suivi de la construction d'une requête avec injection d'un token JWT via un intercepteur, puis un appel POST /search à l'API. Si le token est valide, l'API valide le token via Keycloak, exécute la recherche, et retourne un HTTP 200 avec les données, qui sont traitées et affichées dans une grille. En cas de token invalide, un HTTP 401 est renvoyé, entraînant une redirection vers une page de login pour une nouvelle authentification.

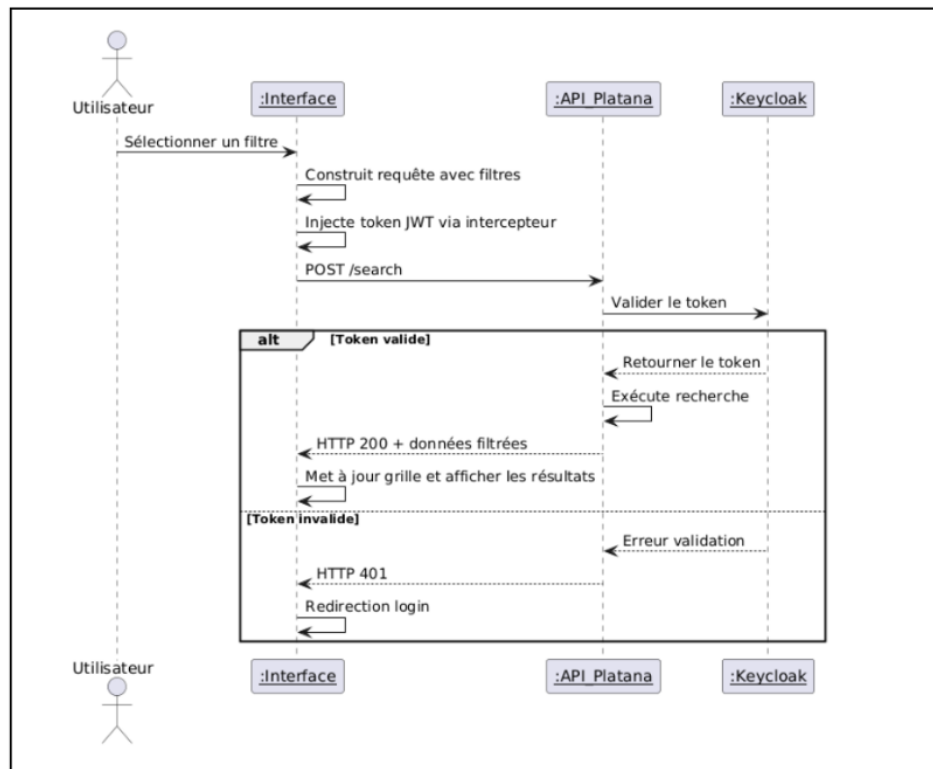


FIGURE 3.9 – Diagramme de séquence du cas d'utilisation « Rechercher des dossiers SAV »

La figure 3.10 illustre le diagramme de séquence du cas d'utilisation «Naviguer dans les résultats». Lorsqu'un utilisateur change de page, une requête POST est envoyée à l'API avec le numéro de page. Après validation du token via Keycloak, l'API retourne les données, qui sont mises à jour et affichées dans l'interface.

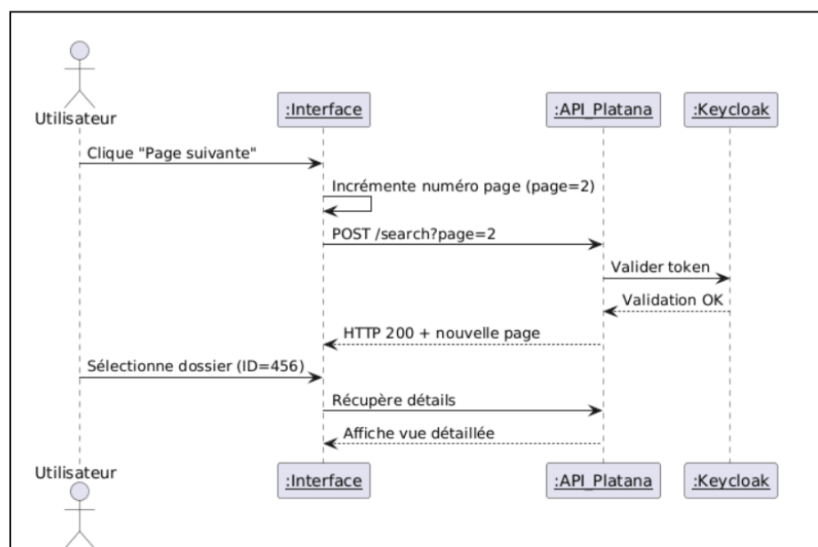


FIGURE 3.10 – Diagramme de séquence du cas d'utilisation « Naviguer dans les résultats »

La figure 3.11 présente le diagramme de séquence du cas « Personnaliser l'interface ». L'utilisateur ouvre le sélecteur de colonnes, choisit les colonnes à afficher, puis la configuration est sauvegardée dans le localStorage et appliquée à la grille.

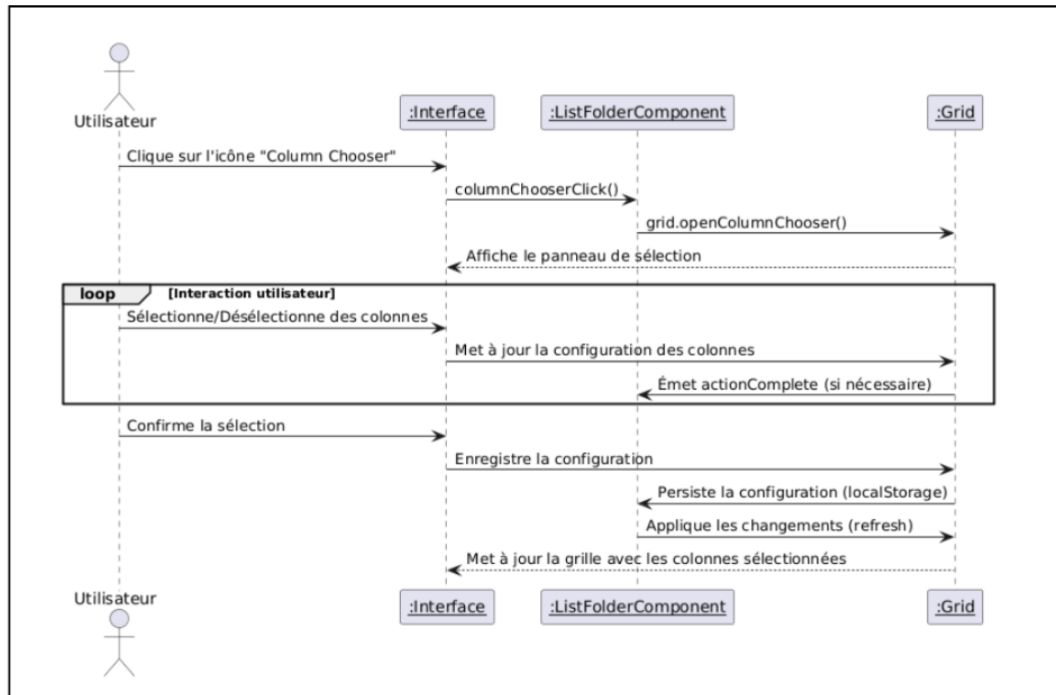


FIGURE 3.11 – Diagramme de séquence du cas d'utilisation «Personnaliser l'interface»

La figure 3.12 illustre le cas «Accéder à la page de création des dossiers». Un clic sur «Créer» déclenche l'ouverture de la page via ListFolderComponent, qui appelle l'API pour récupérer les détails nécessaires à l'affichage du page de création des dossiers.

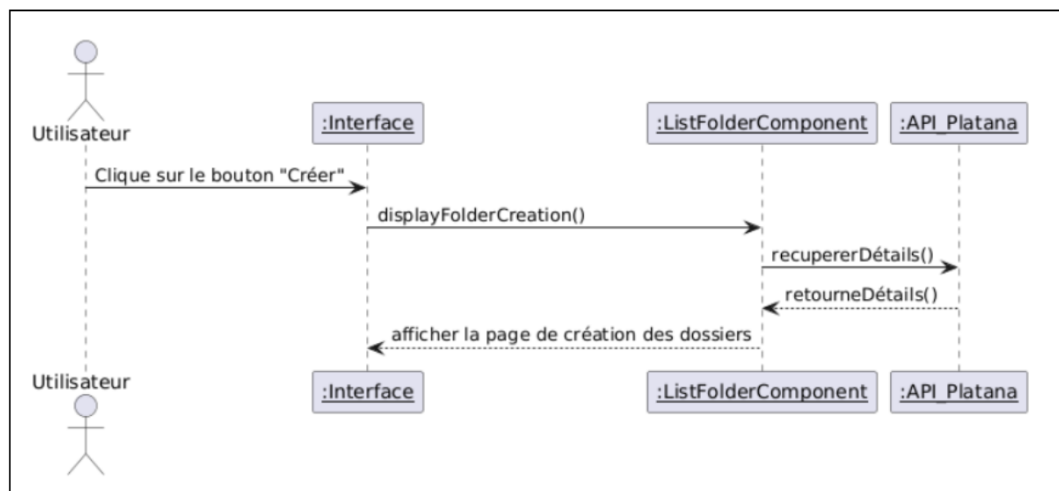


FIGURE 3.12 – Diagramme de séquence du cas d'utilisation «Accéder à la page de création des dossiers»

3.3.4 Diagramme de classes du sprint 2

Le diagramme de classes correspondant au sprint 2, illustré dans la figure 3.13, présente les mêmes classes que celles du sprint 1, à savoir Viewer, Role, Organization, Dossiers SAV et Image produit. Dans ce sprint, de nouvelles fonctionnalités ont été ajoutées et sont représentées en rouge, notamment la classe Admin avec sa méthode displayFolderCreation(), ainsi que les méthodes de la classe Viewer mises en évidence en rouge : clearFieldSearch(), goToNextPage(), goToPreviousPage(), onPageNumberChange(), applyCurrentFilters() et columnChooserClick(args). Ces ajouts enrichissent les capacités de gestion et de navigation dans le système des dossiers SAV.

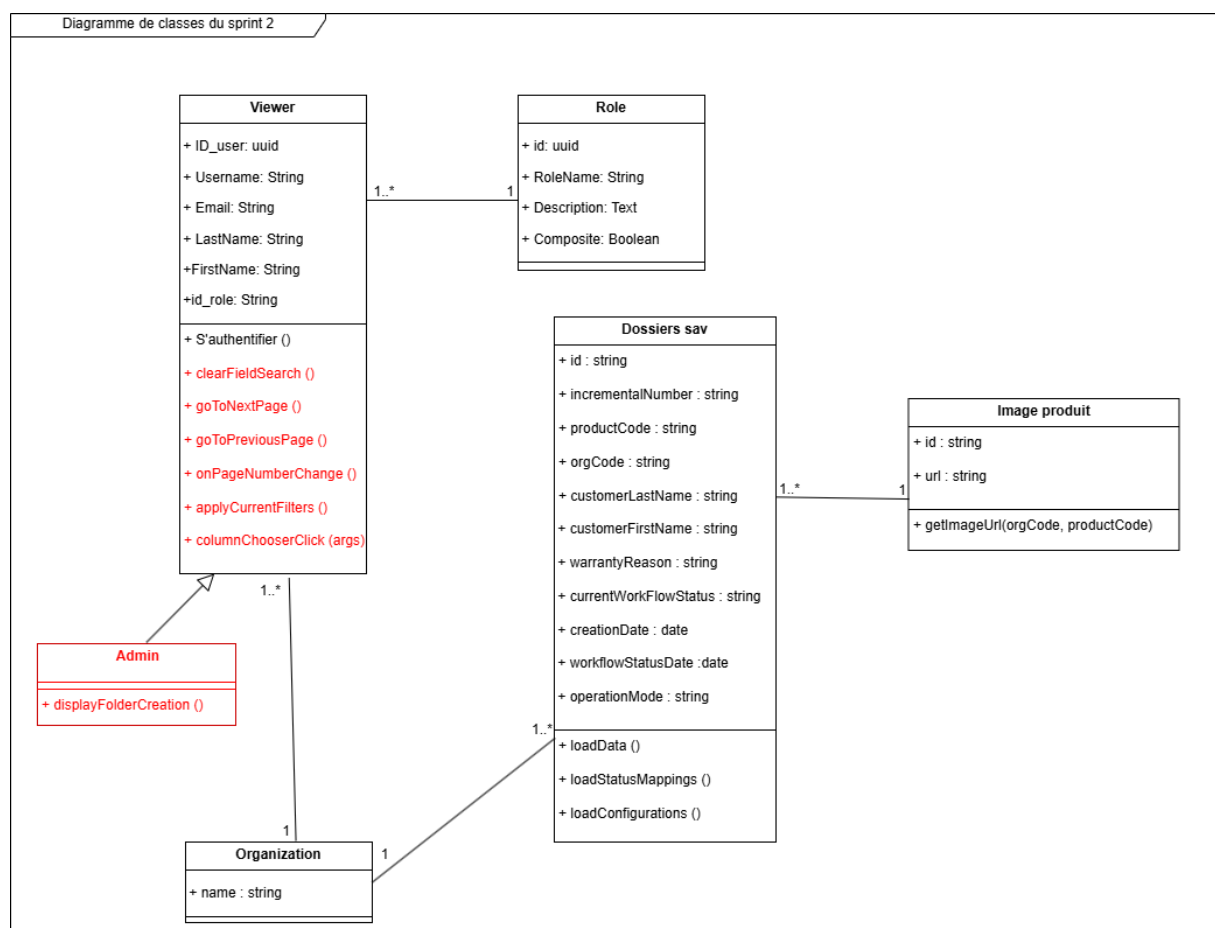


FIGURE 3.13 – Diagramme de classes du sprint 2

3.3.5 Réalisation

Cette section présente les fonctionnalités mises en œuvre lors du deuxième sprint, afin de fournir une meilleure compréhension des tâches effectuées. Les livrables suivants sont réalisés

avec succès.

La figure 3.14 illustre l’interface principale permettant à l’utilisateur d’effectuer des recherches sur les dossiers SAV. Elle offre une vue d’ensemble structurée et intuitive des champs de recherche disponibles.

Platana Dashboard Dossiers Créer Recherche FR Yasmine Ayedi (NR France)							
Liste des dossiers Recherche Garantie Raison de garantie Solution proposée Ouverts Plus Réinitialiser							
NUMERO DOSSIER	PRODUIT	NOM	PRÉNOM	CRÉATION	STATUT	DERNIÈRE ACTIVITÉ	RAISON DE GARANTIE
7074	Vélo électrique de ville WAYSCRAL Hybrid ...	abdelhadi	faten	15/05/2024	Accord de réparation dépassé	Il y a 10 ans	En période de garantie constructeur
7073	Vélo électrique de ville WAYSCRAL Hybrid ...	test	test	15/05/2024	En attente d'enlèvement produit	Il y a 10 ans	Durée de garantie dépassée
7072	Vélo électrique de ville WAYSCRAL Hybrid ...	zaghdene	yasmine	15/05/2024	En attente d'enlèvement produit	Il y a 10 ans	Durée de garantie dépassée
7071	Vélo électrique de ville WAYSCRAL Hybrid ...	AYEDI	AMAL	15/05/2024	Palement magasin	Il y a 10 ans	Durée de garantie dépassée
7069	Vélo électrique de ville WAYSCRAL Hybrid ...	abdelhadi	faten	15/05/2024	En attente du client pour récupère...	Il y a 10 ans	En période de garantie constructeur
7068	Vélo électrique de ville WAYSCRAL Hybrid ...	abdelhadi	faten	15/05/2024	Echange à effectuer pour le client	Il y a 10 ans	En période de garantie constructeur
7066	Vélo électrique de ville WAYSCRAL Hybrid ...	zaghdene	yasmine	14/05/2024	Accord de réparation dépassé	Il y a 10 ans	En période de garantie constructeur
7065	Vélo électrique de ville WAYSCRAL Hybrid ...	abdelhadi	fate,	14/05/2024	Echange à effectuer pour le client	Il y a 10 ans	En période de garantie constructeur
7062	Vélo électrique de ville WAYSCRAL Hybrid ...	abdelhadi	faten	14/05/2024	En cours d'acheminement	Il y a 10 ans	Durée de garantie dépassée
7061	Vélo électrique de ville WAYSCRAL Hybrid ...	abdelhadi	faten	14/05/2024	QUOTATION_ACCEPTATION_DIST...	Il y a 10 ans	Durée de garantie dépassée
Page 11 de 17 Taille de la page 10							

FIGURE 3.14 – Interface de recherche des dossiers

Dans la figure 3.15, – Recherche par numéro de dossier Nous pouvons constater que l’utilisateur peut effectuer une recherche à l’aide d’un numéro de dossier spécifique en utilisant la barre de recherche. Cette fonctionnalité permet d’accéder rapidement à un dossier précis.

Platana Dashboard Dossiers Créer Recherche FR Yasmine Ayedi (NR France)							
Liste des dossiers YASMINE Garantie Raison de garantie Solution proposée Ouverts Plus Réinitialiser							
NUMERO DOSSIER	PRODUIT	NOM	PRÉNOM	CRÉATION	STATUT	DERNIÈRE ACTIVITÉ	RAISON DE GARANTIE
8174	Vélo électrique de ville WAYSCRAL Hybrid ...	ZAGHDENE	YASMINE	17/06/2025	Choix du mode opératoire	Il y a 2 semaines	En période d'extension de garantie
8145	Vélo électrique de ville WAYSCRAL Hybrid ...	ZAGHDENE	YASMINE	23/04/2025	En cours de réparation	Il y a 1 an	En période d'extension de garantie
8027	Vélo électrique de ville WAYSCRAL Hybrid ...	ZAGHDENE	YASMINE	04/02/2025		Il y a 3 ans	En période de garantie constructeur
8026	Vélo électrique de ville WAYSCRAL Hybrid ...	ZAGHDENE	YASMINE	04/02/2025		Il y a 3 ans	En période de garantie constructeur
7825	Vélo électrique de ville WAYSCRAL Hybrid ...	ZAGHDENE	YASMINE	20/09/2024	En cours de réparation	Il y a 7 ans	En période de garantie constructeur
7824	Vélo électrique de ville WAYSCRAL Hybrid ...	ZAGHDENE	YASMINE	20/09/2024	Attente acceptation devis client	Il y a 7 ans	Durée de garantie dépassée
7823	Vélo électrique de ville WAYSCRAL Hybrid ...	ZAGHDENE	YASMINE	20/09/2024	Saisie de besoin pour la réparation	Il y a 7 ans	En période de garantie constructeur
7822	Vélo électrique de ville WAYSCRAL Hybrid ...	ZAGHDENE	YASMINE	20/09/2024	En cours de réparation	Il y a 7 ans	En période de garantie constructeur
7821	Vélo électrique de ville WAYSCRAL Hybrid ...	ZAGHDENE	YASMINE	20/09/2024	En cours de réparation	Il y a 7 ans	En période de garantie constructeur
7807	Vélo électrique de ville WAYSCRAL Hybrid ...	ZAGHDENE	YASMINE	20/09/2024	Attente réception pièces dépassé	Il y a 3 ans	En période de garantie constructeur
Page 1 de 2 Taille de la page 10							

FIGURE 3.15 – Interface de recherche intégrant une barre de saisie

La figure 3.16 présente la recherche de dossiers à l’aide de filtres avancés. L’exemple illustré utilise le filtre « solution proposée » pour affiner la recherche selon le mode opératoire appliqué.

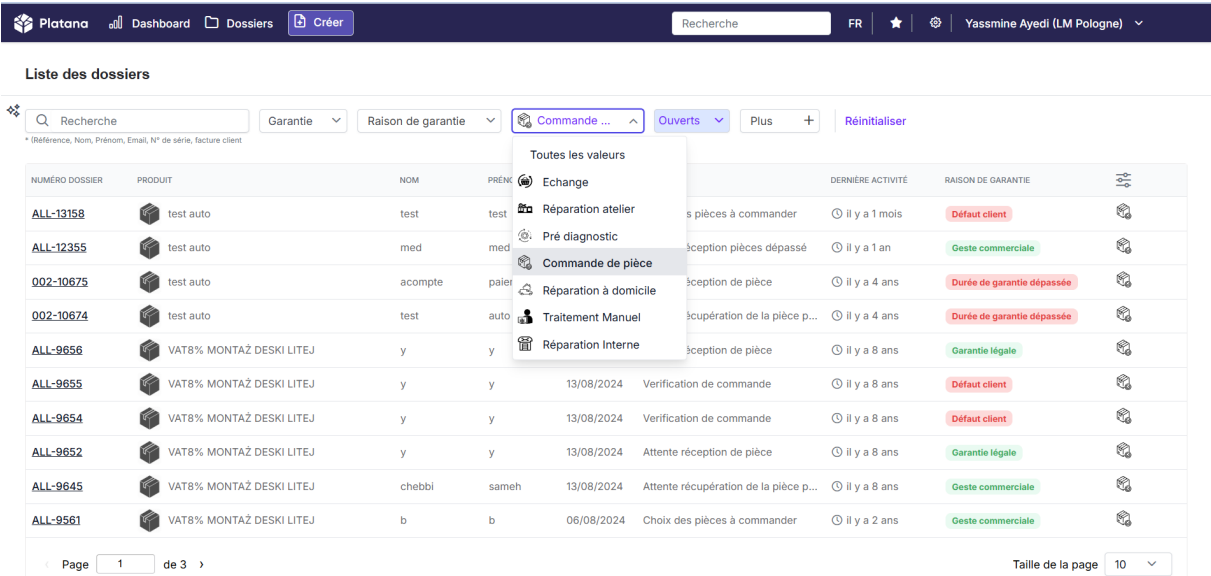


FIGURE 3.16 – Interface de recherche avec filtres avancés

La figure 3.17 illustrée ici permet à l’utilisateur de personnaliser l’affichage de la grille, en ajoutant ou en supprimant des colonnes selon ses préférences, pour une expérience de consultation sur mesure.

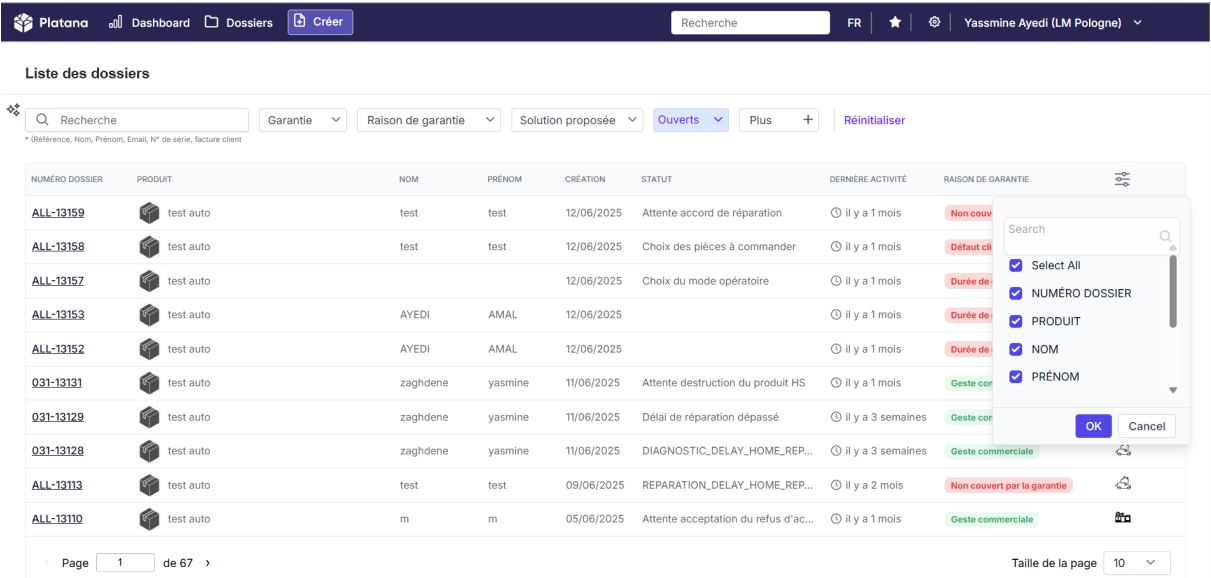


FIGURE 3.17 – Personnalisation de la grille

La figure 3.18 met en évidence le système "Plus" pour les filtres additionnels, offrant aux utilisateurs la possibilité d'accéder à plusieurs autres filtres en cliquant sur un menu déroulant "Plus".

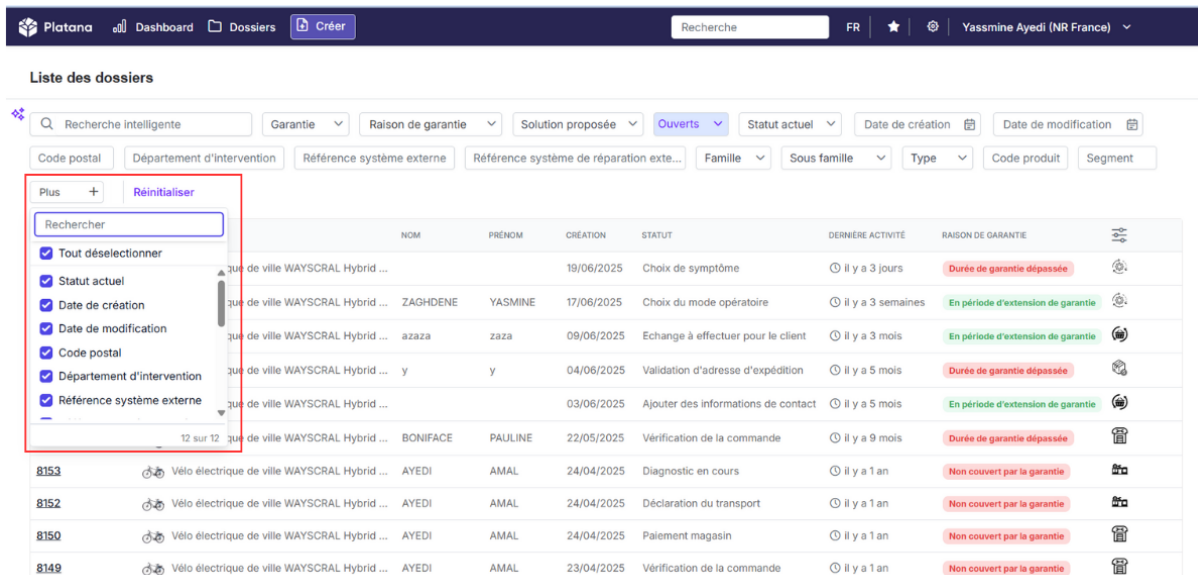


FIGURE 3.18 – Interface de sélection des filtres additionnels via 'Plus'

3.4 Conclusion

Ce chapitre a présenté les progrès réalisés lors des sprints 1 et 2, qui ont établi les fondations fonctionnelles de l'application SAV de Platana. Le sprint 1 a permis la mise en place d'une authentification sécurisée avec Keycloak et d'un affichage optimisé via Syncfusion, tandis que le sprint 2 a enrichi l'interface avec des fonctionnalités de recherche, navigation et personnalisation, améliorant ainsi l'ergonomie et la flexibilité. Ces étapes ont posé une base robuste et intuitive pour les développements futurs. Le chapitre suivant est dédié à la mise en œuvre de la recherche intelligente et à la personnalisation avancée de l'application.

Mise en œuvre de la recherche intelligente et personnalisation de l'application SAV

Sommaire

4.1	Introduction	53
4.2	Sprint 3 : Mise en place de l'intelligence artificielle pour la recherche des dossiers SAV	53
4.2.1	Backlog du sprint 3	53
4.2.2	Identification des plateformes d'intégration des modèles LLM .	54
4.2.3	Présélection des modèles LLM pour la recherche intelligente . .	55
4.2.4	Configuration du prompt engineering	55
4.2.5	Tests réalisés et choix du modèle LLM	56
4.2.6	Paramètres ajustables du modèle retenu	57
4.2.7	Tests réalisés et choix de la plateforme d'intégration API	58
4.2.8	Résultats obtenus du modèle LLaMA 70B-8192	59
4.3	Sprint 4 : Intégration de la recherche intelligente et optimisation multilingue	61
4.3.1	Backlog du sprint 4	61
4.3.2	Diagramme du cas d'utilisation du sprint 4	62
4.3.3	Diagrammes de séquence du sprint 4	65
4.3.4	Diagramme de classes du sprint 4	68
4.3.5	Réalisation	68
4.4	Conclusion	70

4.1 Introduction

Ce chapitre détaille les avancées réalisées lors des sprints 3 et 4 dans le cadre du développement de l'application SAV de Platana, avec un accent particulier sur l'intégration d'une recherche intelligente et l'optimisation de l'expérience utilisateur. Le sprint 3 s'est concentré sur la mise en place d'une recherche sémantique basée sur l'intelligence artificielle, en explorant différents modèles de langage et en sélectionnant des plateformes d'intégration performantes. Le sprint 4 a permis d'achever cette intégration, en combinant la recherche IA avec des filtres manuels, en implémentant une prise en charge multilingue et en développant la fonctionnalité de basculement entre organisations.

4.2 Sprint 3 : Mise en place de l'intelligence artificielle pour la recherche des dossiers SAV

Cette section est dédiée au troisième sprint, qui porte sur la mise en place d'une intelligence artificielle (IA) pour la recherche des dossiers SAV, une étape cruciale visant à transformer l'efficacité opérationnelle de l'application SAV de Platana.

4.2.1 Backlog du sprint 3

Le tableau 4.1 détaille le backlog du sprint 3.

TABLE 4.1 – Backlog du sprint 3

Nom du sprint	Tâches
Mise en place de l'intelligence artificielle pour la recherche des dossiers SAV	Exploration des modèles LLM.
	Définition des critères de sélection.
	Tests de performance des modèles.
	Sélection du modèle LLaMA 70B-8192.
	Conception initiale du prompt.

Nom du sprint	Tâches
	Tests unitaires du prompt.

Dans ce sprint, les sections suivantes détaillent les étapes de mise en œuvre de la recherche intelligente. Tout d'abord, nous présentons la présélection de deux plateformes permettant l'intégration de modèles LLM via API. Ensuite, nous abordons la présélection de deux modèles LLM issus de ces plateformes, cette sélection étant fondée sur des critères tels que le multilinguisme, ainsi que les performances sémantiques et contextuelles pour la compréhension des requêtes en langage naturel. Nous décrivons la configuration du prompt engineering, suivie de tests unitaires visant à évaluer la compatibilité des modèles avec les requêtes en langage naturel et la génération de filtres JSON. Une fois un modèle retenu, nous ajustons ses paramètres pour optimiser ses performances, puis nous évaluons les plateformes Groq Cloud et OpenRouter sur ce même modèle, en nous concentrant sur leurs capacités à offrir des API performantes, avec une attention particulière portée au temps de réponse. Enfin, nous présentons les résultats obtenus avec le modèle choisi, illustrés par des exemples concrets démontrant ses performances.

4.2.2 Identification des plateformes d'intégration des modèles LLM

Dans le cadre du sprint 3, consacré à l'intégration d'une recherche intelligente pour les dossiers SAV, l'objectif était d'identifier des plateformes capables d'intégrer des modèles de traitement du langage naturel (LLM) via une API.

Deux plateformes ont été retenues à l'issue de cette analyse :

- **Groq [5]** : proposée par Groq Inc., cette plateforme d'inférence IA repose sur un matériel propriétaire (LPU – Language Processing Unit) permettant une exécution ultra-rapide des modèles LLM.



FIGURE 4.1 – Logo Groq

- **OpenRouter [20]** : un agrégateur d'API IA qui permet de se connecter à un large éventail de modèles (OpenAI, Anthropic, Meta, Mistral, etc.) via une interface unifiée. Il facilite l'expérimentation grâce à sa flexibilité et sa compatibilité avec des formats d'API standards.



FIGURE 4.2 – Logo OpenRouter

4.2.3 Présélection des modèles LLM pour la recherche intelligente

À partir des deux plateformes identifiées précédemment, deux modèles LLM, assurant une bonne compréhension sémantique des requêtes en langage naturel dans plusieurs langues (français, anglais, allemand, espagnol, etc.), ont été présélectionnés pour des tests approfondis.

- LaMA 70B-8192 [4] : développé par Meta AI, ce modèle est reconnu pour ses capacités de compréhension multilingue et sa fenêtre contextuelle étendue à 8192 tokens. Il se montre performant dans des tâches telles que la classification, le résumé, et l'interprétation de prompts complexes.
- DeepSeek R1 Distill LLaMA 70B [21] : un modèle open-source dérivé de LLaMA 3, optimisé pour le raisonnement logique. Il se distingue par une capacité contextuelle atteignant 128 000 tokens, le rendant particulièrement adapté aux tâches structurées (conversion de formats, traitement de données, mappage de concepts techniques).

4.2.4 Configuration du prompt engineering

Le prompt engineering [22] désigne l'ensemble des techniques visant à optimiser la formulation des entrées fournies aux modèles de langage pour obtenir des sorties précises et fiables.

Dans le cadre de l'application SAV, cette technique est essentielle pour permettre aux modèles LLaMA3-70B-8192 et DeepSeek R1 Distill LLaMA 70B de traduire des requêtes en langage naturel en filtres JSON compatibles avec l'API /search. Le prompt a été conçu pour garantir :

- Normalisation des sorties : Conversion fiable des termes multilingues en filtres standardisés (par exemple, « sous garantie » → « SG », « échange » → « SWAP »).
- Réduction des incohérences : Utilisation de règles explicites et d'exemples concrets pour encadrer les réponses et éviter les hallucinations.
- Compatibilité JSON : Production d'un format JSON strict, directement exploitable par le backend.
- Support multilingue : Prise en charge des requêtes dans plusieurs langues (par exemple, « warranty dossiers requiring swap » en anglais, « Garantieunterlagen für Austausch » en allemand).

4.2.5 Tests réalisés et choix du modèle LLM

Des tests approfondis ont été conduits pour évaluer les performances des modèles LLaMA3-70B-8192 et DeepSeek R1 Distill Llama 70B dans le contexte de la recherche intelligente.

Pour la requête « dossiers sous garantie nécessitant échange », LLaMA3-70B-8192 a généré une réponse JSON stricte et correcte, comme illustré dans la figure 4.3 .



```
1 {
2   "searchValue": null,
3   "filters": {
4     "closed": {"value": false, "operator": "EQUAL"},
5     "newWarranty": {"value": "SG", "operator": "EQUAL"},
6     "operationMode": {"value": "SWAP", "operator": "EQUAL"}
7   },
8   "page": 0,
9   "size": 10
10 }
```

FIGURE 4.3 – Réponse du modèle LLaMA3-70B-8192

Ce résultat démontre la fiabilité du modèle pour produire des filtres conformes sans post-traitement. Le prompt a correctement mappé « sous garantie » à « SG » et « échange » à « SWAP », respectant les règles de structure JSON et de pagination.

En revanche, DeepSeek R1 Distill Llama 70B a produit une réponse correcte mais non conforme au format JSON strict, incluant du texte narratif, comme illustré la figure 4.4 .

En-têtes	Charge utile	Aperçu	Réponse	Initiateur	Délai
			<pre>I also remember that by default, the closed filter should be set to false, so I'll include that as well. Now, putting it all together, the filters will include closed, newWarranty, and operationMode. The searchValue I should also make sure to include the pagination parameters, page: 0 and size: 10, as per the rules. So, the final JSON should look like this with all the filters and pagination included. </think> ```json { "searchValue": null, "filters": { "closed": {"value": false, "operator": "EQUAL"}, "newWarranty": {"value": "SG", "operator": "EQUAL"}, "operationMode": {"value": "SWAP", "operator": "EQUAL"} }, "page": 0, "size": 10 } ... </pre>		

FIGURE 4.4 – Réponse du modèle DeepSeek R1 Distill Llama 70B

Cette limitation, rend DeepSeek moins adapté au projet, malgré ses atouts en raisonnement. LLaMA3-70B-8192 est retenu comme modèle principal pour sa fiabilité et son intégration directe.

4.2.6 Paramètres ajustables du modèle retenu

Après avoir sélectionné le modèle LLaMA3-70B-8192 pour implémenter la recherche intelligente des dossiers SAV, nous avons choisi d'optimiser ses performances et sa précision à l'aide de paramètres ajustables. Ces paramètres, permettent d'adapter le comportement du modèle aux contraintes spécifiques du projet. Les deux paramètres principaux sont les suivants :

- Max Tokens [23] : Ce paramètre détermine le nombre maximal de jetons que le modèle peut générer dans une réponse. Dans un premier temps, la valeur avait été fixée à 1024, mais elle a ensuite été réduite à 512 à la suite de tests empiriques. Ces tests ont montré que même pour des requêtes complexes comportant plusieurs filtres les réponses JSON générées restaient généralement en dessous de 200 jetons. La limite à 512 jetons offre ainsi une marge de sécurité suffisante pour éviter toute troncature, tout en réduisant la latence et les coûts de traitement.
- Temperature [24] : Le paramètre temperature contrôle la variabilité ou la créativité des réponses produites par le modèle. Il a été fixé à 0,1, une valeur volontairement basse, afin d'assurer des réponses déterministes, précises et cohérentes. Ce réglage est essentiel pour

garantir que le modèle produise des filtres JSON stricts, en adéquation avec les règles de transformation définies dans le prompt (par exemple : « sous garantie » → « SG », ou la conversion des dates en timestamps). Des valeurs plus élevées (comme 0,7 ou 1,0) favoriseraient une production plus créative, mais augmenteraient également le risque de réponses incohérentes ou non conformes au format JSON, ce qui serait incompatible avec les contraintes de l'API /search.

Ces paramètres, combinés à un prompt bien conçu, assurent une intégration robuste et performante du modèle dans le système SAV.

4.2.7 Tests réalisés et choix de la plateforme d'intégration API

Après avoir sélectionné LLaMA3-70B-8192 comme modèle principal et ajusté ses paramètres (max tokens : 512, temperature : 0.1), des tests comparatifs ont été réalisés pour évaluer les performances des plateformes Groq et OpenRouter lors de l'intégration via leurs APIs respectives. Les tests ont consisté à soumettre des requêtes identiques, telles que « dossiers sous garantie nécessitant échange », et à mesurer la latence totale ainsi que la qualité des réponses JSON. Les résultats ont montré une latence moyenne de 0,4 seconde pour Groq, comme illustré dans la figure 4.5 .

CREATED	MODEL	API KEY	CODE	LATENCY	REQUEST ID
29/05/2025 16:05:17	llama3-70b-8192	key_0...62jm	200	0.384	req_0...mm1x
29/05/2025 16:00:47	llama3-70b-8192	key_0...62jm	200	0.363	req_0...w8mb
29/05/2025 14:54:08	llama3-70b-8192	key_0...62jm	200	0.294	req_0...kp34
29/05/2025 14:54:05	llama3-70b-8192	key_0...62jm	200	0.299	req_0...gvb7

FIGURE 4.5 – Logs de génération du modèle LLaMA3-70B-8192 via Groq

Cette performance, combinée à des réponses JSON strictes, fait de Groq la plateforme préférée pour une recherche intelligente en temps réel.

En revanche, OpenRouter a affiché une latence moyenne de 5 secondes, comme indiqué dans la figure 4.6 .

Generation details	
Provider	Chutes
latency	4,53 seconds

FIGURE 4.6 – Logs de génération du modèle LLaMA3-70B-8192 via openRouter

Cette différence s'explique par l'architecture d'OpenRouter, qui repose sur des fournisseurs tiers, introduisant des délais dans la transmission des requêtes et des réponses. De plus, certaines réponses d'OpenRouter incluaient des métadonnées inutiles, nécessitant un post-traitement léger pour une intégration fluide. Ces limitations positionnent Groq comme la solution optimale pour le projet, bien qu'OpenRouter reste une alternative viable pour des cas nécessitant une flexibilité multi-modèles. Les tests confirment la capacité de Groq à répondre aux exigences de latence et de fiabilité, avec une perspective d'évolutivité via son offre payante pour la production.

4.2.8 Résultats obtenus du modèle LLaMA 70B-8192

Cette section expose les résultats de l'évaluation du modèle LLaMA 70B-8192, sélectionné comme solution optimale pour la recherche intelligente des dossiers SAV. Les tests ont révélé une latence moyenne de 0,3 seconde par requête, assurant des réponses rapides et adaptées aux besoins des utilisateurs. Le modèle se distingue par sa robustesse face aux erreurs, sa prise en charge de requêtes multilingues, et son respect des standards JSON ainsi que des mécanismes de pagination.

Des exemples concrets, illustrés par les captures d'écran suivantes, mettent en évidence ces performances .

La figure 4.7 montre une requête utilisateur saisie en français contenant des fautes.

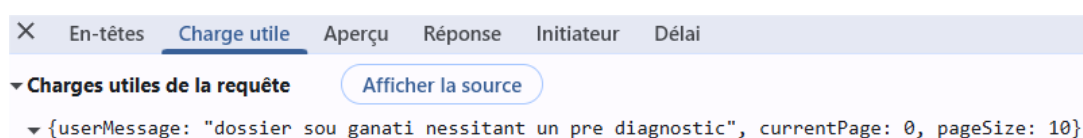


FIGURE 4.7 – Exemple de requête en français avec fautes d'orthographe saisie par un utilisateur

La figure 4.8 présente la réponse du modèle .

```

1  {
2    "searchValue": null,
3    "filters": {
4      "closed": {"value": false, "operator": "EQUAL"},
5      "newWarranty": {"value": "SG", "operator": "EQUAL"},
6      "operationMode": {"value": "PRE_DIAGNOSTIC", "operator": "EQUAL"}
7    },
8    "page": 0,
9    "size": 10
10 }

```

FIGURE 4.8 – Réponse du modèle à une requête en français contenant des fautes d'orthographe

identifié les filtres pertinents. Malgré les erreurs orthographiques, le modèle a correctement
La figure 4.9 illustre une requête multilingue en espagnol.

▼ Charges utiles de la requête [Afficher la source](#)

▼ {userMessage: "Expedientes que tienen una garantía legal y requieren una reparación a domicilio",...}

FIGURE 4.9 – Requête formulée en espagnol par l'utilisateur

La figure 4.10 présente la réponse du modèle .

```

1  {
2    "searchValue": null,
3    "filters": {
4      "closed": {"value": false, "operator": "EQUAL"},
5      "newWarranty": {"value": "SG", "operator": "EQUAL"},
6      "warrantyReason": {"value": "LEGAL_WARRANTY", "operator": "EQUAL"},
7      "operationMode": {"value": "HOME_REPAIR", "operator": "EQUAL"}
8    },
9    "page": 0,
10   "size": 10
11 }

```

FIGURE 4.10 – Réponse du modèle à une requête en espagnol

Cette réponse démontre la capacité du modèle à traiter des langues différentes tout en maintenant une structure JSON cohérente.

La figure 4.11 présente la requête : dossiers créés le 5 mai 2025.

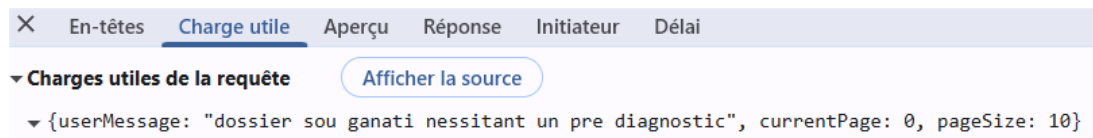


FIGURE 4.11 – Requête demandant les dossiers avec une date de création spécifique

La figure 4.12 affiche la réponse du modèle .



FIGURE 4.12 – Réponse du modèle avec conversion précise de la date en timestamps

Le modèle a converti la date en timestamps (du 5 mai 2025 00 :00 :00.000 à 23 :59 :59.059 GMT+01 :00), témoignant de sa capacité à gérer des critères temporels avec précision.

Ces résultats valident la fiabilité du modèle LLaMA 70B-8192 pour une recherche intelligente robuste, multilingue, et parfaitement adaptée aux exigences des dossiers SAV.

4.3 Sprint 4 : Intégration de la recherche intelligente et optimisation multilingue

Cette section présente le sprint 4, consacré à l'intégration de la recherche intelligente et à l'optimisation multilingue. Ce sprint permet de développer l'API ai-search en Spring Boot, de combiner la recherche classique avec celle basée sur l'IA, d'implémenter la prise en charge multilingue via i18n, et de mettre en place une fonctionnalité de basculement entre organisations. Ces évolutions améliorent l'ergonomie, l'accessibilité et la personnalisation de l'interface utilisateur.

4.3.1 Backlog du sprint 4

Le tableau 4.2 détaille le backlog du sprint 4 .

TABLE 4.2 – Backlog du sprint 4

Nom du sprint	Tâches
Intégration de la recherche intelligente et optimisation multilingue	Développement de l'API ai-search en Spring Boot.
	Création d'un service d'injection des réponses.
	Configuration de la combinaison recherche classique + IA.
	Implémentation de l'internationalisation (i18n).
	Développement de la fonctionnalité de basculement entre les organisations.
	Tests d'intégration et validation multilingue.

4.3.2 Diagramme du cas d'utilisation du sprint 4

Le diagramme de cas d'utilisation de la figure 4.13 illustre les principaux cas d'usage du sprint 4, mettant en évidence les interactions des différents acteurs de l'application. Les acteurs Viewer et Admin peuvent Rechercher les dossiers avec IA, Combiner la recherche intelligente avec les filtres avancées, et Changer la langue de l'interface. Le Super Admin hérite des capacités du Viewer et dispose en outre de la possibilité de Basculer entre les organisations, offrant ainsi une gestion plus étendue du système pour les dossiers SAV.

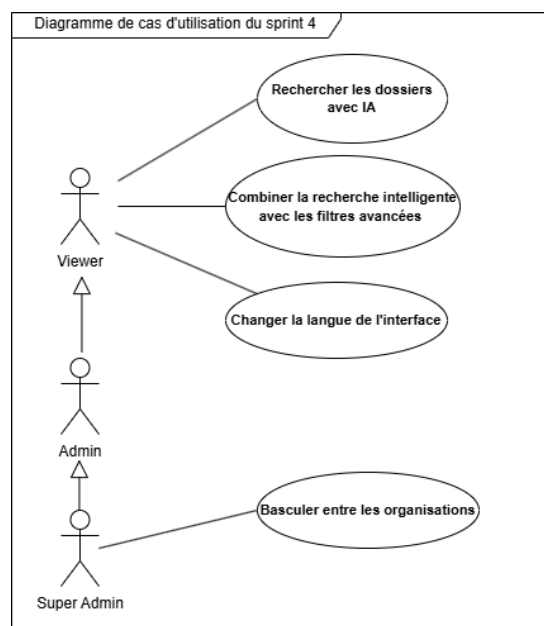


FIGURE 4.13 – Diagramme de cas d'utilisation du sprint 4

Le tableau 4.3 résume la description textuelle du cas d'utilisation «Rechercher les dossiers avec IA».

TABLE 4.3 – Description textuelle du cas d'utilisation « Rechercher les dossiers avec IA »

Titre	Rechercher les dossiers avec IA
Acteur	Super admin, Admin, Viewer
Résumé	L'acteur saisit une requête en langage naturel et le système utilise l'API ai-search pour générer des filtres via LLaMA 70B-8192, affichant les résultats dans la grille Syncfusion.
Pré-condition	L'acteur est authentifié via Keycloak et le modèle LLaMA 70B-8192 est accessible via Groq.
Post-condition	Les résultats sont affichés avec une latence moyenne de 0,3 seconde dans une grille paginée.
Scénario nominal	1. L'acteur accède à la section de recherche de l'interface. 2. L'acteur saisit une requête (ex : "dossiers sous garantie"). 3. L'acteur clique sur le bouton de recherche IA. 4. Le système envoie une requête à l'API ai-search. 5. Le système affiche les résultats paginés dans la grille Syncfusion.
Scénario alternatif	A1 : Connexion à Groq échouée 1. L'API Groq est indisponible. 2. Le système affiche un message d'erreur (ex. : "Service IA indisponible"). 3. Le scénario reprend au point 2 du scénario nominal.

Le tableau 4.4 résume la description textuelle du cas d'utilisation "Combiner la recherche intelligente avec des filtres avancés" .

TABLE 4.4 – Description textuelle du cas d'utilisation « Combiner la recherche intelligente avec des filtres avancés »

Titre	Combiner la recherche intelligente avec des filtres avancés
Acteur	Super admin, Admin, Viewer

Résumé	L'acteur saisit une requête en langage naturel et applique des filtres manuels, combinant les filtres IA générés par LLaMA 70B-8192 avec des filtres classiques, puis affiche les résultats paginés dans la grille Syncfusion.
Pré-condition	L'acteur est authentifié via Keycloak. Le modèle LLaMA 70B-8192 est accessible via Groq.
Post-condition	Les résultats combinés sont affichés dans une grille paginée.
Scénario nominal	1. L'acteur accède à la section de recherche de l'interface. 2. L'acteur saisit une requête (ex. : "dossiers sous garantie"). 3. L'acteur sélectionne des filtres via dropdowns (ex. : statut "en cours"). 4. Le système envoie une requête à l'API ai-search avec les filtres combinés. 5. Le système affiche les résultats paginés dans la grille Syncfusion.
Scénario alternatif	A1 : Filtres invalides 1. L'acteur sélectionne des filtres incompatibles (ex : date de création future). 2. La grille affiche «Aucun résultat à afficher». 3. Le scénario reprend au point 3 du scénario nominal.

Le tableau 4.5 résume la description textuelle du cas d'utilisation « Changer la langue de l'interface».

TABLE 4.5 – Description textuelle du cas d'utilisation « Changer la langue de l'interface »

Titre	Changer la langue de l'interface
Acteur	Super admin, Admin, Viewer
Résumé	L'acteur sélectionne une langue via l'AppBar, et le système met à jour les libellés et messages de l'interface en utilisant i18n.
Pré-condition	L'acteur est authentifié via Keycloak et les fichiers de traduction sont chargés.

Post-condition	L'interface est mise à jour dans la langue choisie.
Scénario nominal	1. L'acteur accède à l'interface principale. 2. L'acteur clique sur le dropdown de langue dans l'AppBar. 3. L'acteur sélectionne une langue (ex : anglais). 4. Le système met à jour les libellés (ex : "Rechercher" devient "Search").

Le tableau 4.6 résume la description textuelle du cas d'utilisation Basculer entre les organisations.

TABLE 4.6 – Description textuelle du cas d'utilisation « Basculer entre les organisations »

Titre	Basculer entre les organisations
Acteur	Super admin
Résumé	Le Super admin sélectionne une organisation via l'AppBar, et le système affiche les dossiers SAV associés à cette organisation.
Pré-condition	Le Super admin est authentifié via Keycloak.
Post-condition	L'interface affiche les dossiers SAV de l'organisation choisie.
Scénario nominal	1. Le Super admin accède à l'interface principale. 2. Le Super admin clique sur le menu déroulant des organisations dans l'AppBar. 3. Le Super admin sélectionne une organisation (ex. : LM France). 4. Le système affiche les dossiers SAV liés à l'organisation sélectionnée.

4.3.3 Diagrammes de séquence du sprint 4

Le diagramme de séquence du cas «Recherche avec filtre IA» montre comment une requête utilisateur est traitée par l'API ai-search, qui interroge le modèle LLaMA 70B-8192 via Groq pour générer des filtres JSON. Ces filtres sont injectés dans l'API search, qui retourne des résultats paginés à l'interface.

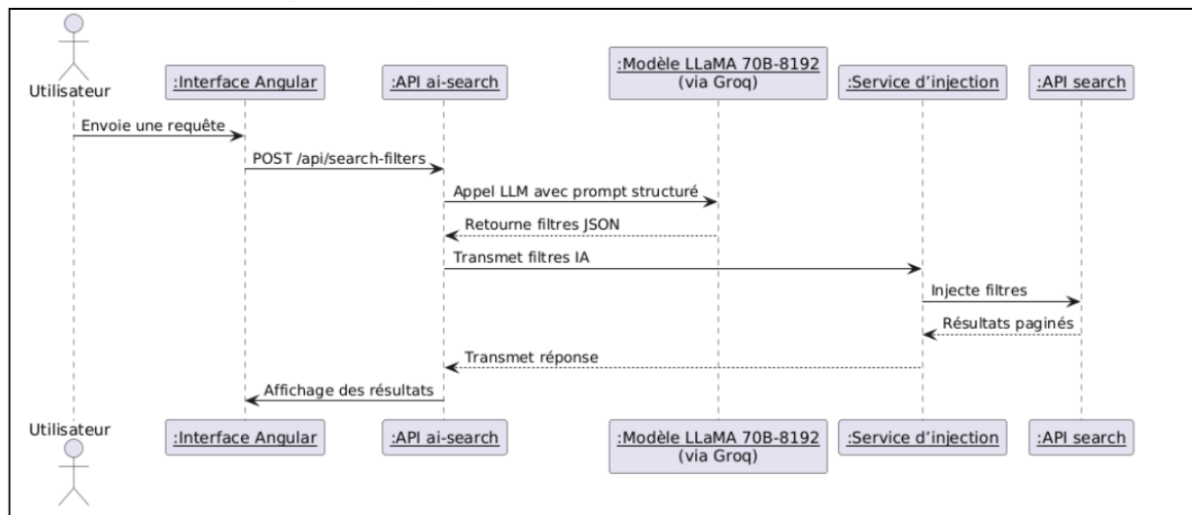


FIGURE 4.14 – Diagramme de séquence du cas d'utilisation «Recherche avec filtre IA»

La figure 4.15 présente le diagramme de séquence du cas d'utilisation «Combinaison de recherche intelligente et filtres classiques». Elle décrit la fusion entre les filtres manuels saisis par l'utilisateur et ceux générés via l'IA. L'API ai-search interroge le modèle, puis les résultats sont combinés par un service d'injection avant d'être transmis à l'API search, qui retourne des résultats paginés à l'interface.

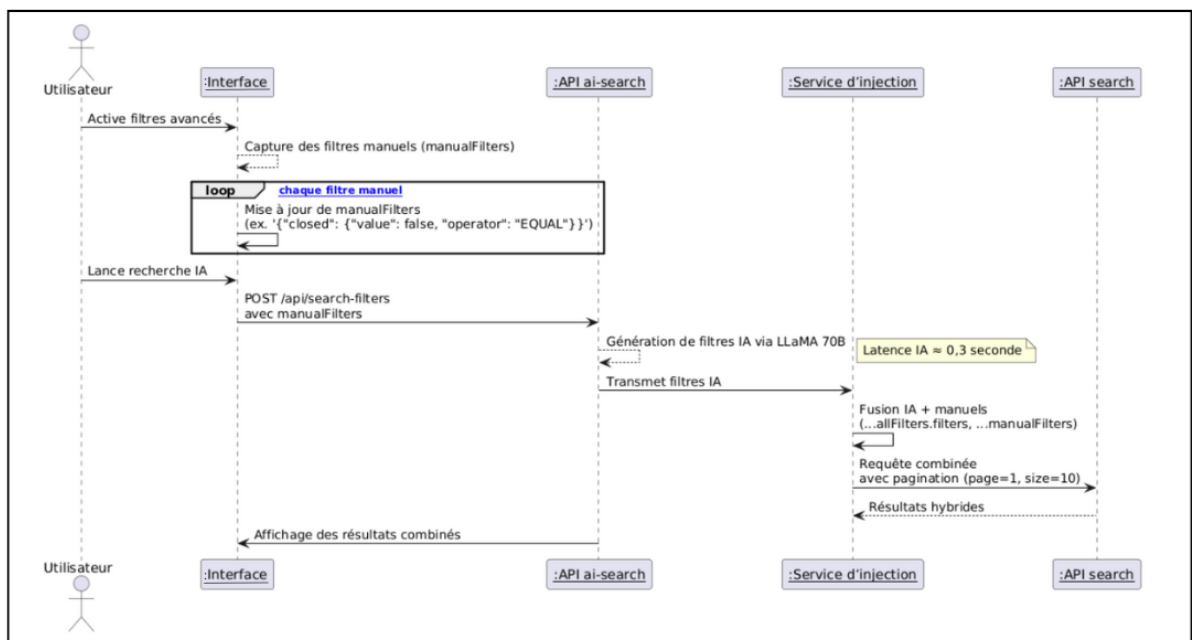


FIGURE 4.15 – Diagramme de séquence du cas d'utilisation «Combinaison de recherche intelligente et filtres classiques»

La figure 4.16, représentant le diagramme de séquence du cas d'utilisation «Changer la langue de l'interface». Après la sélection d'une langue, le TranslateService met à jour les libellés,

émet l'événement langChanged, puis les composants sont actualisés selon la langue choisie.

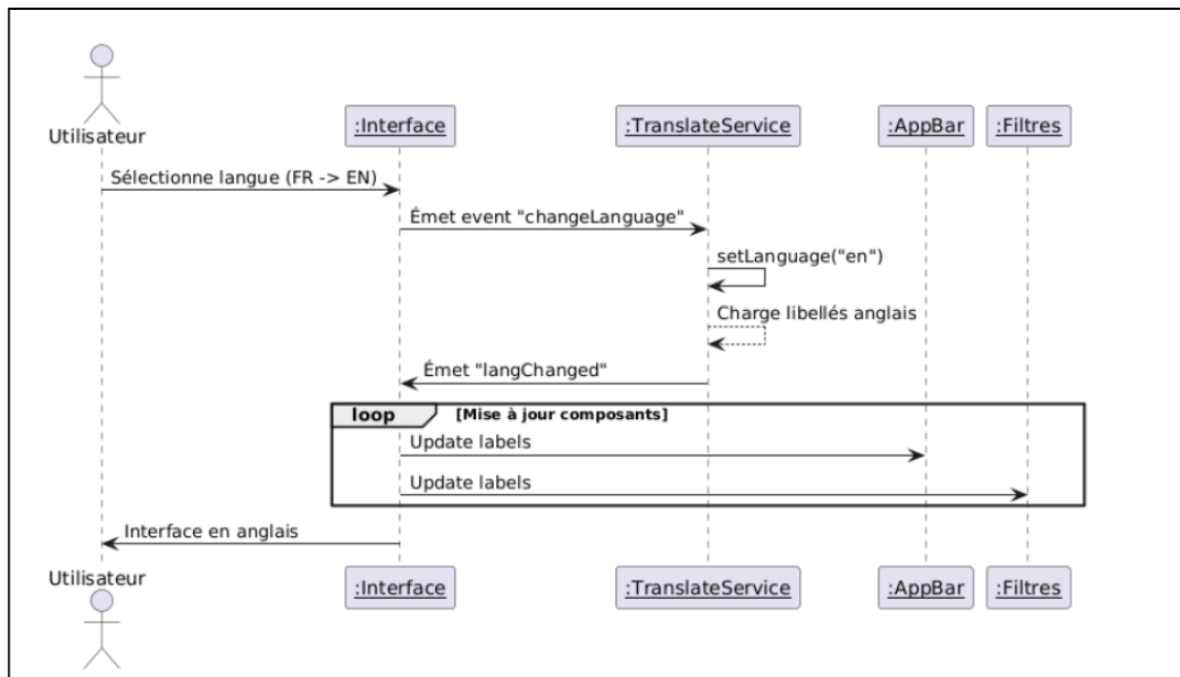


FIGURE 4.16 – Diagramme de séquence du cas d'utilisation «Changer la langue de l'interface»

La figure 4.17 illustre le cas d'utilisation «Basculer entre les organisations». Le Super Admin sélectionne une organisation, AppBarComponent transmet cette sélection à OrganizationService, qui met à jour currentOrganization dans les services abonnés. L'API retourne alors les données associées et les affiche dans l'interface.

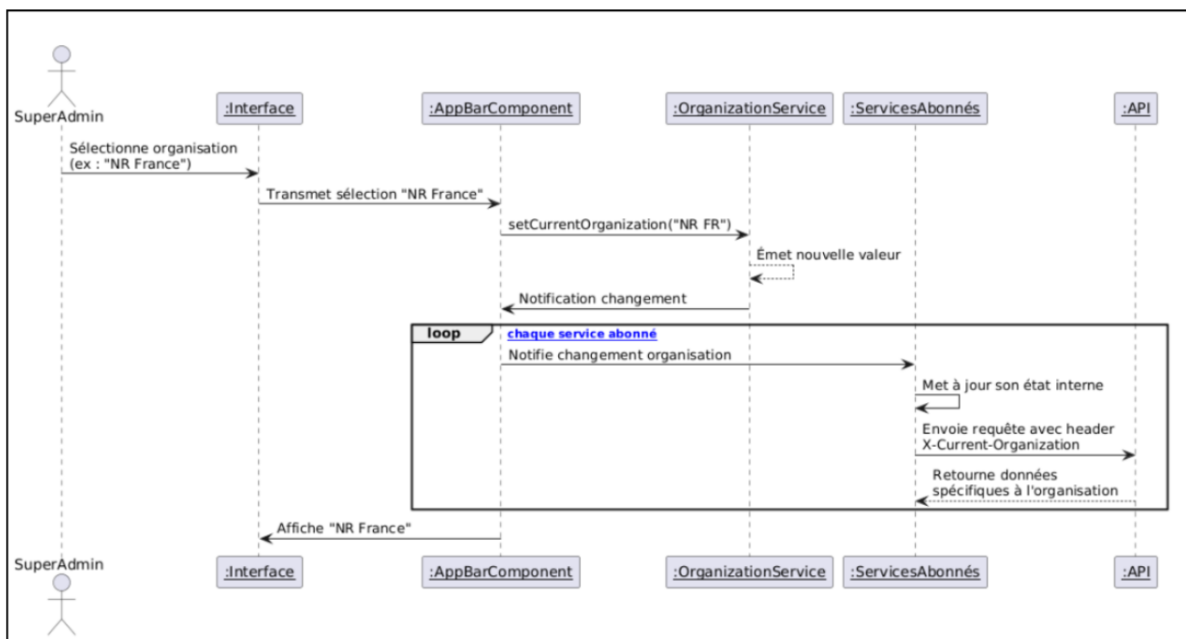


FIGURE 4.17 – Diagramme de séquence du cas d'utilisation «Basculer entre les organisation»

4.3.4 Diagramme de classes du sprint 4

La figure 4.18 illustre le diagramme de classes des entités principales, reprenant celles du sprint 2 : Viewer, Role, Organization, Dossiers SAV et ImageProduit. De nouvelles fonctionnalités sont ajoutées en rouge, telles que la classe SuperAdmin avec la méthode onOrganizationSelect, ainsi que les méthodes aiSearchRequest(), toggleAISearch() et switchLang() dans la classe Viewer, ce qui renforce les capacités de recherche intelligente et de personnalisation linguistique du système de gestion des dossiers SAV.

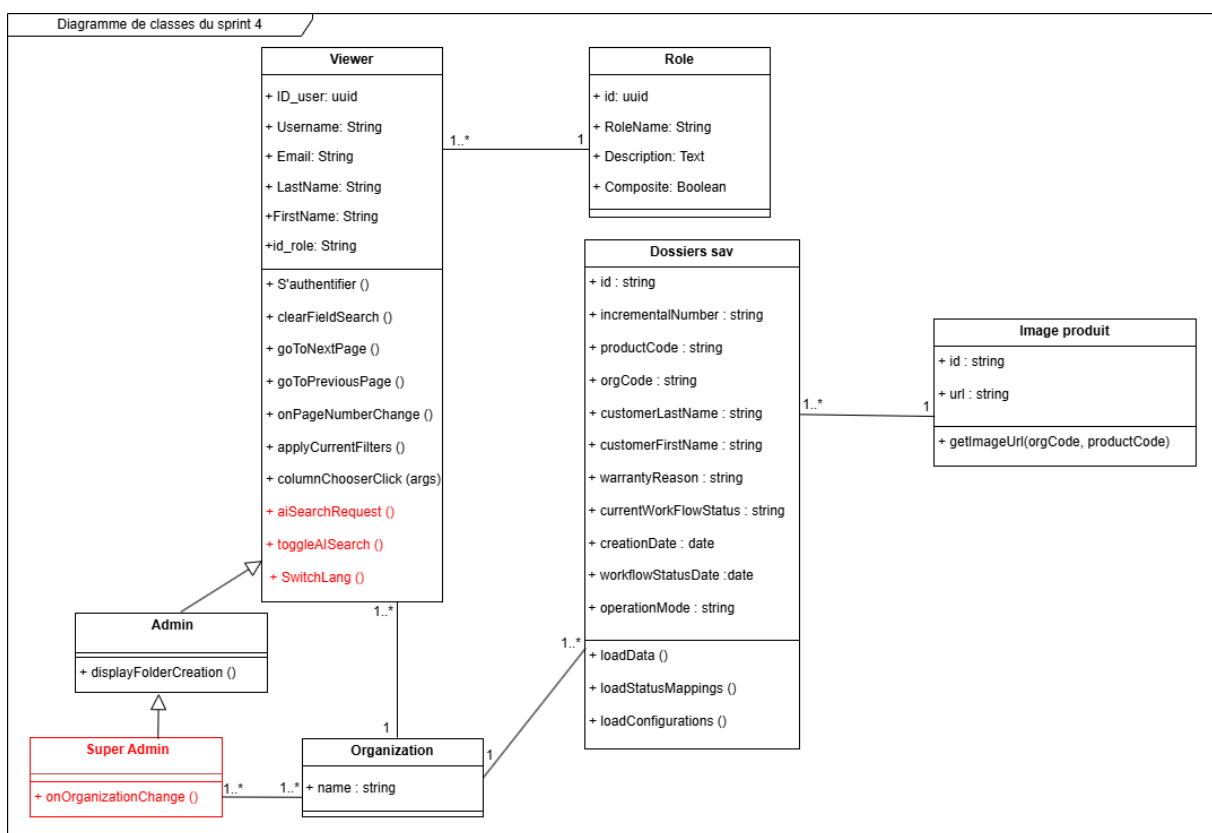


FIGURE 4.18 – Diagramme de classes du sprint 4

4.3.5 Réalisation

Cette section présente les livrables du sprint 4, validés avec succès. L'API ai-search développée en Spring Boot, a été implémentée afin de traduire les requêtes en langage naturel en filtres JSON, en s'appuyant sur le modèle LLaMA 70B-8192, avec une latence moyenne de 0,3 seconde. Un service d'injection a également été mis en place pour intégrer ces filtres dans

l'API search, permettant ainsi une combinaison fluide entre la recherche classique (barre de recherche, filtres avancés) et la recherche assistée par IA.

L'implémentation de l'internationalisation (i18n) a permis de changer la langue de l'interface, avec des libellés dynamiques gérés via l'AppBar. Enfin, une fonctionnalité a été développée pour permettre à l'utilisateur de basculer entre différentes organisations de manière intuitive. La figure 4.19 illustre l'interface de combinaison de recherche intelligente et filtres classiques :

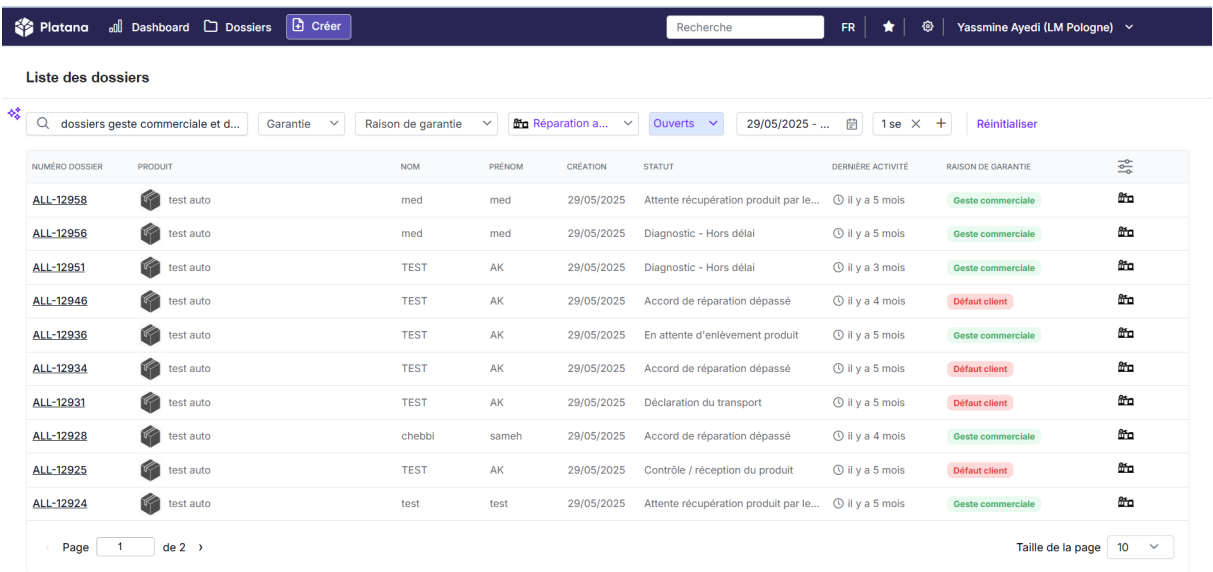


FIGURE 4.19 – Interface de combinaison de recherche intelligente et filtres classiques

La figure 4.20 montre l'interface de sélection de la langue, permettant à l'utilisateur de basculer dynamiquement entre différentes langues.

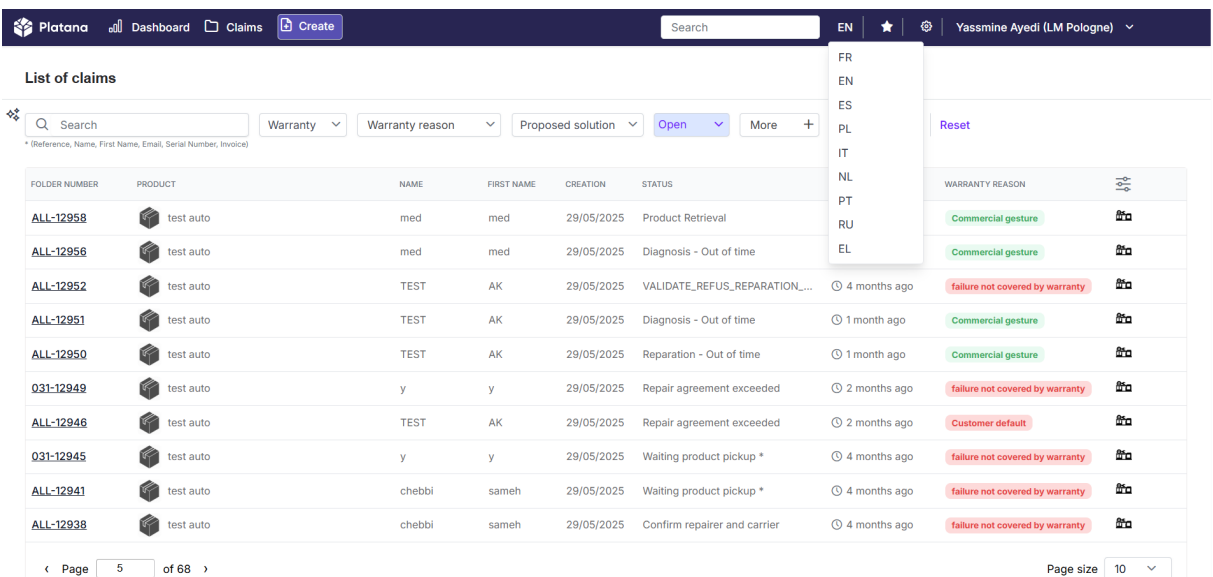


FIGURE 4.20 – Interface de sélection de la langue

La figure 4.21 montre l'interface de sélection des organisations, permettant au Super Admin de choisir dynamiquement une organisation parmi celles disponibles.

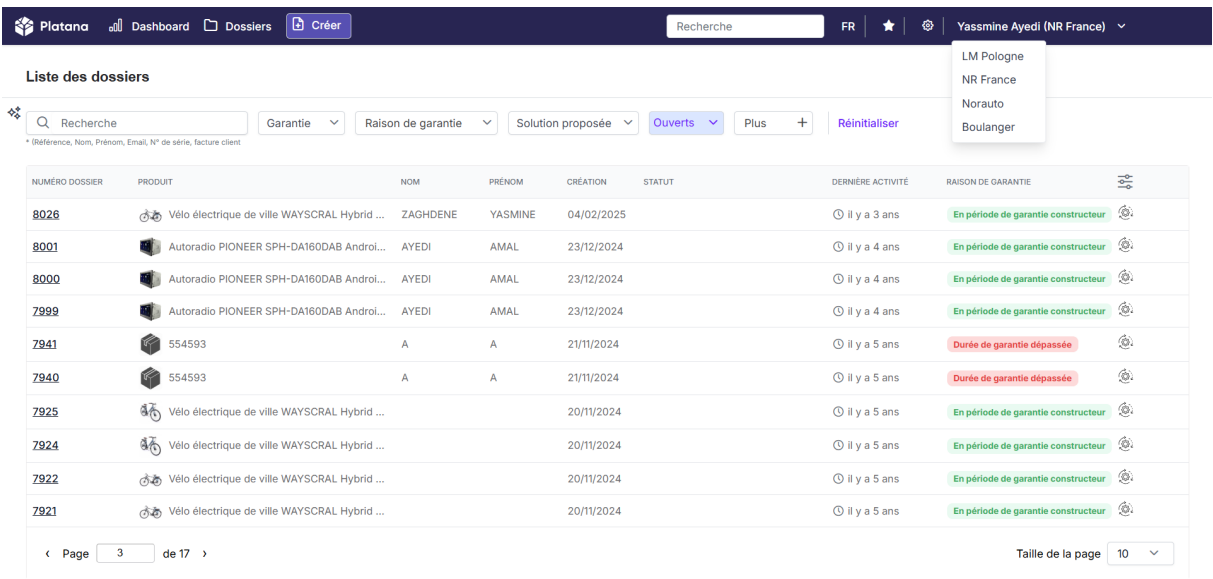


FIGURE 4.21 – Interface de sélection des organisations

4.4 Conclusion

Ce chapitre a marqué une étape décisive dans le développement de l'application SAV de Platana, avec la mise en œuvre réussie de la recherche intelligente et des fonctionnalités de personnalisation lors des sprints 3 et 4. Le sprint 3 a permis l'intégration d'un moteur sémantique basé sur le modèle LLaMA 70B-8192, optimisé via Groq pour une latence moyenne de 0,3 seconde, et capable de gérer des requêtes multilingues avec robustesse. Le sprint 4 a consolidé ces avancées en combinant la recherche IA avec des filtres manuels, en implémentant l'internationalisation via i18n, et en ajoutant des options de basculement entre organisations, améliorant ainsi l'ergonomie et l'accessibilité. Ces résultats démontrent une solution performante et adaptable, prête à évoluer pour répondre aux besoins croissants des opérateurs SAV.



CONCLUSION GÉNÉRALE

Ce travail a été réalisé dans le cadre de mon projet de fin d'études en vue de l'obtention du diplôme d'ingénieur en électronique et communication. Il s'est déroulé au sein de l'entreprise Platana, où nous avons conçu et développé un module de recherche intelligente pour les dossiers SAV. Au cours de ce stage, nous avons modernisé l'interface existante en la refondant avec Angular 18 et Syncfusion, tout en intégrant une recherche sémantique basée sur le modèle LLaMA 70B-8192 via l'API Groq, répondant ainsi aux besoins d'efficacité et d'ergonomie des opérateurs.

Ce projet a nécessité de relever plusieurs défis, notamment l'adaptation d'une architecture obsolète à une technologie moderne, l'optimisation des performances pour garantir une latence inférieure à 0,5 seconde, et la gestion des complexités liées à l'intégration d'une intelligence artificielle multilingue. Ces contraintes, associées à des délais serrés, ont constitué des enjeux majeurs tout au long du développement.

Au terme de ce travail, nous pouvons dire que cette expérience a été globalement enrichissante. Elle nous a permis d'appliquer nos connaissances théoriques dans un contexte professionnel concret, au sein d'un environnement motivant où notre contribution a été reconnue. Nous sommes satisfaits des résultats obtenus, notamment d'avoir livré une interface intuitive, une recherche efficace et une prise en charge multilingue, qui devraient représenter un atout pour Platana et ses clients.

Bien que cette solution réponde aux exigences fonctionnelles et non fonctionnelles identifiées, elle offre encore des perspectives d'amélioration. Par exemple, nous envisageons d'ajouter

une fonctionnalité permettant aux utilisateurs d'enregistrer leurs filtres les plus utilisés ou de créer des favoris personnalisés. Cette option permettrait une personnalisation accrue, facilitant l'accès rapide aux recherches fréquentes et améliorant l'expérience des opérateurs SAV. De plus, une autre perspective prometteuse réside dans l'intégration d'une recherche assistée par vocal, exploitant des technologies de reconnaissance vocale avancées pour permettre aux opérateurs de formuler des requêtes naturelles par la voix. Cette fonctionnalité, combinée à l'intelligence artificielle existante, pourrait accélérer les interactions, réduire la dépendance au clavier, et offrir une interface plus accessible, notamment dans des environnements bruyants ou pour des utilisateurs ayant des besoins spécifiques.



WEBOGRAPHIE

- [1] **PLATANA** : <https://www.platana.fr/qui-sommes-nous>, visité le 5 février 2025.
- [2] **Angular** : <https://www.simplilearn.com/tutorials/angular-tutorial/what-is-angular>, visité le 6 février 2025.
- [3] **syncfusion** : <https://www.syncfusion.com/company/about-us> , visité le 6 février 2025.
- [4] **LLaMA 70B-8192**, : <https://console.groq.com/docs/model/llama3-70b-8192> , visité le 25 Avril 2025.
- [5] **Groq** : <https://groq.com/> , visité le 17 Avril 2025.
- [6] **Angular 10** : <https://v10.angular.io/docs> , visité le 6 février 2025.
- [7] **material UI** : <https://mui.com/material-ui/getting-started/> , visité le 6 février 2025.
- [8] **LLM** : <https://www.cloudflare.com/fr-fr/learning/ai/what-is-large-language-model/> visité le 14 Mars 2025.
- [9] **scrum** : <https://ignition-program.com/tuto/la-methode-scrum-pour-les-nuls>, visité le 10 février 2025.
- [10] **Diagramme de Gantt** : <https://www.gantt.com/creating-gantt-charts>, visité le 25 Mai 2025.
- [11] **IntelliJ** : <https://www.simplilearn.com/what-is-intellij-idea-article> , visité le 10 février 2025.
- [12] **Postman** : <https://www.postman.com/product/what-is-postman/>,, visité le 12 février 2025.
- [13] **ts** : <https://www.typescriptlang.org/> , visité le 10 février 2025.
- [14] **Keycloak** : <https://www.keycloak.org/> , visité le 5 février 2025.
- [15] **Spring Boot** : <https://www.ibm.com/fr-fr/topics/java-spring-boot>

- [16] **draw.io** : <https://appsource.microsoft.com/en-us/product/office/wa200003444?tab=overview> , visité le 15 mars 2025.
- [17] **gitlab** : <https://www.geeksforgeeks.org/gitlab/>, visité le 1 mars 2025.
- [18] **i18n** : <https://phrase.com/blog/posts/i18n-a-simple-definition/> , visité le 1 juin 2025.
- [19] **NER** : <https://www.ibm.com/fr-fr/think/topics/named-entity-recognition> , visité le 12 Mars 2025.
- [20] **OpenRouter** : <https://openrouter.ai/docs/quickstart> , visité le 14 Avril 2025.
- [21] **DeepSeek R1 Distill Llama 70B** : <https://lambda.ai/inference-models/deepseek-llama3.3-70b> , visité le 25 Avril 2025.
- [22] **Prompt engineering** : <https://www.salesforce.com/fr/resources/definition/prompt-engineering/> visité le 20 Avril 2025.
- [23] **Max tokens** : <https://medium.com/@ramakrishna.etneni/introduction-to-max-tokens-its-usage-and-limitations-c76c26351017>, visité le 25 Avril 2025.
- [24] **Temperature** : <https://www.ibm.com/think/topics/llm-temperature>, visité le 25 Avril 2025.

Refonte du Back Office Frontend avec Développement d'un Module de Recherche Intelligente

Yessmine Ayédi

Résumé :

Ce projet de fin d'études a pour objectif de moderniser la gestion des dossiers du SAV en procédant à une refonte complète de l'interface existante et en intégrant une recherche intelligente basée sur l'intelligence artificielle. La solution développée s'appuie sur une modernisation technologique avec Angular 18 et Syncfusion, combinée à une recherche sémantique alimentée par le modèle LLaMA 70B-8192 via l'API Groq, offrant une interface intuitive, rapide et multilingue. Les résultats obtenus mettent en évidence une amélioration notable de l'ergonomie, une réduction des temps de recherche grâce à une compréhension sémantique des requêtes, et une adaptabilité aux besoins d'une clientèle internationale.

Mots clés : Angular 18, Syncfusion, Spring Boot, LLM, Keycloak, LLaMA 70B-8192, i18n.

Abstract :

This final-year project aims to modernize the management of after-sales service files by completely redesigning the existing interface and integrating intelligent search based on artificial intelligence. The developed solution leverages modern technologies such as Angular 18 and Syncfusion, combined with semantic search powered by the LLaMA 70B-8192 model via the Groq API, providing an intuitive, fast, and multilingual interface. The results highlight significant improvements in ergonomics, reduced search times through semantic understanding of queries, and adaptability to the needs of an international clientele.

Key-words : Angular 18, Syncfusion, Spring Boot, LLM, Keycloak, LLaMA 70B-8192, i18n .

تلخيص:

هذا المشروع الختامي، الذي تم إنجازه داخل شركة PLATANA يهدف إلى تحديث إدارة ملفات خدمة ما بعد البيع من خلال إعادة تصميم شاملة للواجهة الحالية، بالإضافة إلى دمج بحث ذكي يعتمد على تقنيات الذكاء الاصطناعي. تعتمد الحلول المطورة على تحديث تقني باستخدام Angular18 و Syncfusion إلى جانب بحث دلالي مدعوم بنموذج LLaMA 70B-8192 عبر واجهة برمجة التطبيقات الخاصة بـ Groq، مما يوفر واجهة استخدام سلسة، سريعة و متعددة اللغات. تظهر النتائج المحققة تحسناً ملحوظاً في سهولة الاستخدام ، وتقليصاً في أوقات البحث بفضل الفهم الدلالي للطلبات، بالإضافة إلى قابلية التكيف مع احتياجات العملاء الدوليين .

الكلمات المفتاحية: Angular 18، Syncfusion، Spring Boot، النماذج اللغوية الكبيرة (LLM)،

Keycloak، LLaMA 70B-8192، التدويل (i18n).

